

无监督学习





- 无监督学习
- 无监督特征学习
 - 主成分分析
 - 稀疏编码
 - 自编码器
 - 稀疏自编码器
 - 降噪自编码器
- 概率密度估计
 - 参数密度估计
 - 非参数密度估计：核方法、K近邻方法



» 监督学习

- 建立映射关系 $f: x \rightarrow y$

» 无监督学习

- 指从无标签的数据中学习出一些有用的模式。
- 聚类：建立映射关系 $f: x \rightarrow y$
(不借助于任何人工给出标签或者反馈等指导信息)
- 特征学习
- 密度估计 $p(x)$

聚类

Unsupervised Learning



Cluster 1

$$\begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix}$$



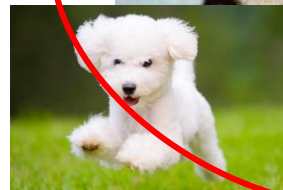
Cluster 3

$$\begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

Open question: how many clusters do we need?

$$\begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix}$$

Cluster 2



- K-means

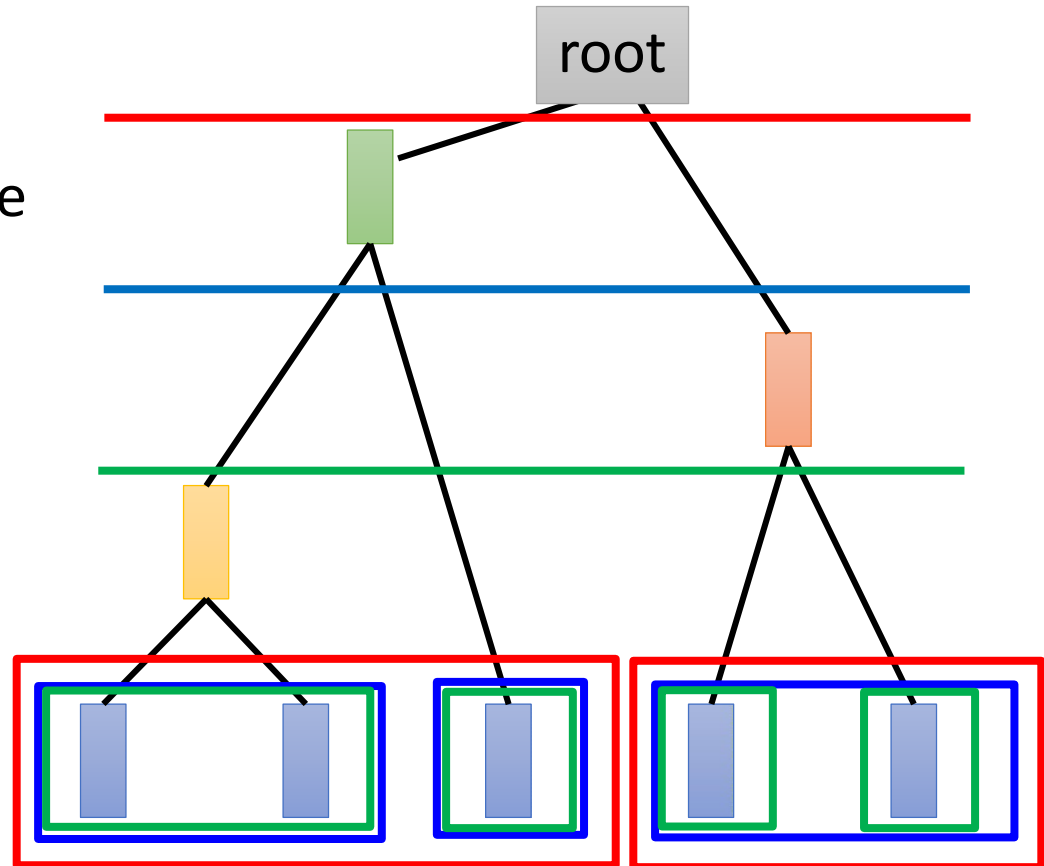
- Clustering $X = \{x^1, \dots, x^n, \dots, x^N\}$ into K clusters
- Initialize cluster center c^i , $i=1, 2, \dots, K$ (K random x^n from X)
- Repeat
 - For all x^n in X :
$$b_i^n = \begin{cases} 1 & x^n \text{ is most "close" to } c^i \\ 0 & \text{Otherwise} \end{cases}$$
 - Updating all c^i :
$$c^i = \sum_{x^n} b_i^n x^n / \sum_{x^n} b_i^n$$



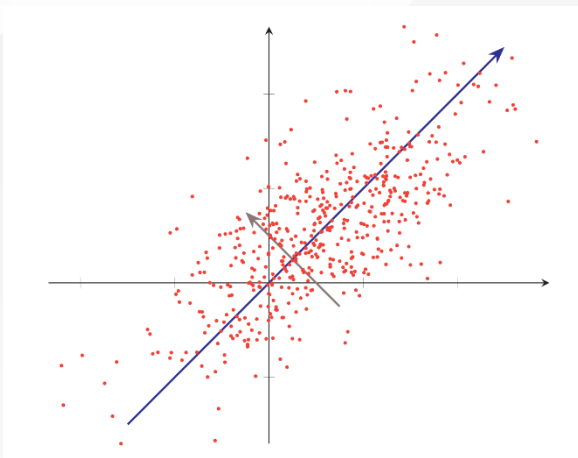
- Hierarchical Agglomerative Clustering (HAC)

Step 1: build a tree

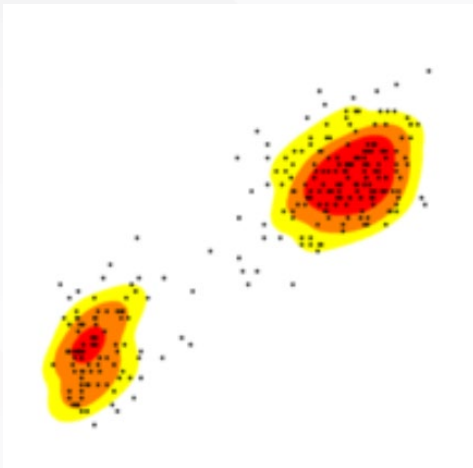
Step 2: pick a threshold



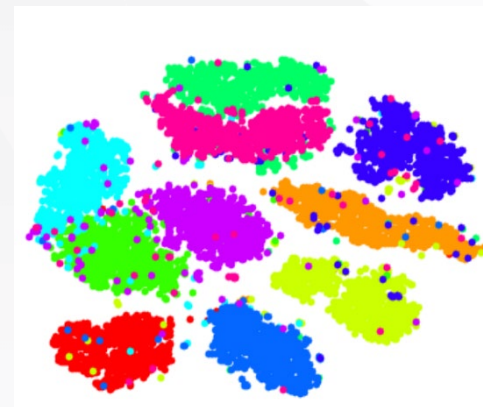
典型的无监督学习问题



无监督特征学习



密度估计



聚类

为什么要无监督学习？



大脑有大约 10^{14} 个突触，我们只能活大约 10^9 秒。所以我们有比数据更多的参数。这启发了我们必须进行大量无监督学习的想法，因为感知输入（包括本体感受）是我们可以获得每秒 10^5 维约束的唯一途径。

-- Geoffrey Hinton, 2014 AMA on Reddit

主成分分析

Principal Component Analysis, PCA



一种最常用的数据降维方法，使得在转换后的空间中数据的方差最大。

- 样本点 $\mathbf{x}^{(n)}$ 投影之后的表示为

$$\mathbf{z}^{(n)} = \mathbf{w}^\top \mathbf{x}^{(n)}$$

- 所有样本投影后的方差为

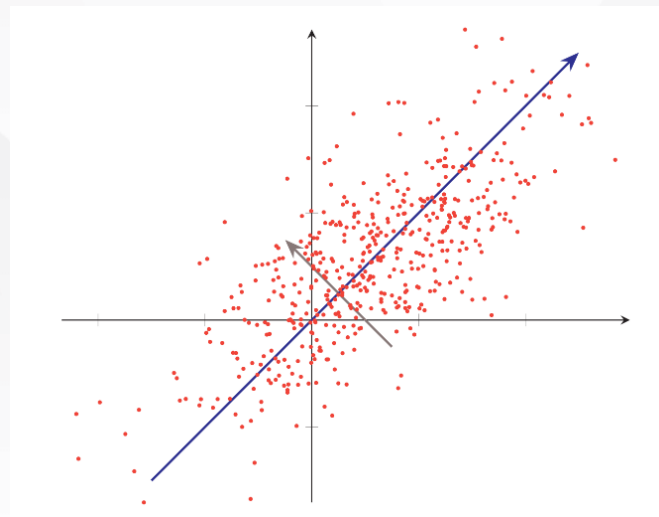
$$\begin{aligned}\sigma(\mathbf{X}; \mathbf{w}) &= \frac{1}{N} \sum_{n=1}^N (\mathbf{w}^\top \mathbf{x}^{(n)} - \mathbf{w}^\top \bar{\mathbf{x}})^2 \\ &= \frac{1}{N} (\mathbf{w}^\top \mathbf{X} - \mathbf{w}^\top \bar{\mathbf{X}})(\mathbf{w}^\top \mathbf{X} - \mathbf{w}^\top \bar{\mathbf{X}})^\top \\ &= \mathbf{w}^\top \Sigma \mathbf{w},\end{aligned}$$

- 目标函数

$$\max_{\mathbf{w}} \mathbf{w}^\top \Sigma \mathbf{w} + \lambda(1 - \mathbf{w}^\top \mathbf{w})$$

对目标函数求导并令导数等于0，可得

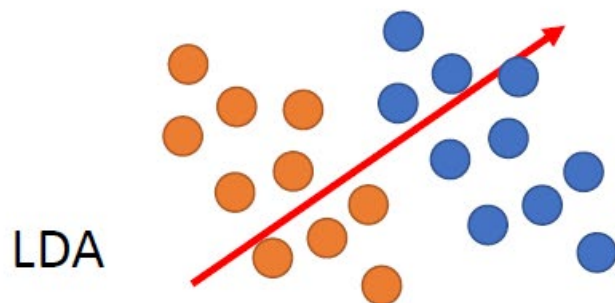
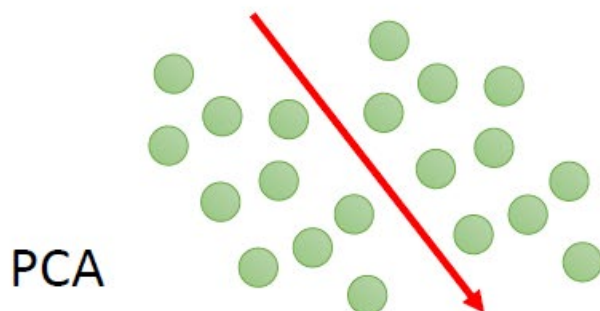
$$\Sigma \mathbf{w} = \lambda \mathbf{w}$$



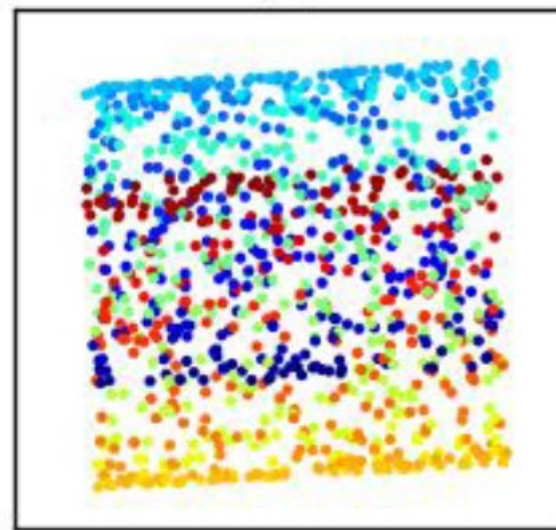
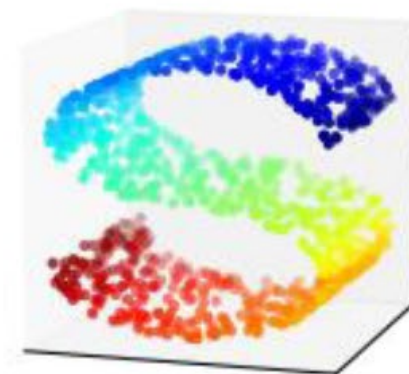


Weakness of PCA

- Unsupervised



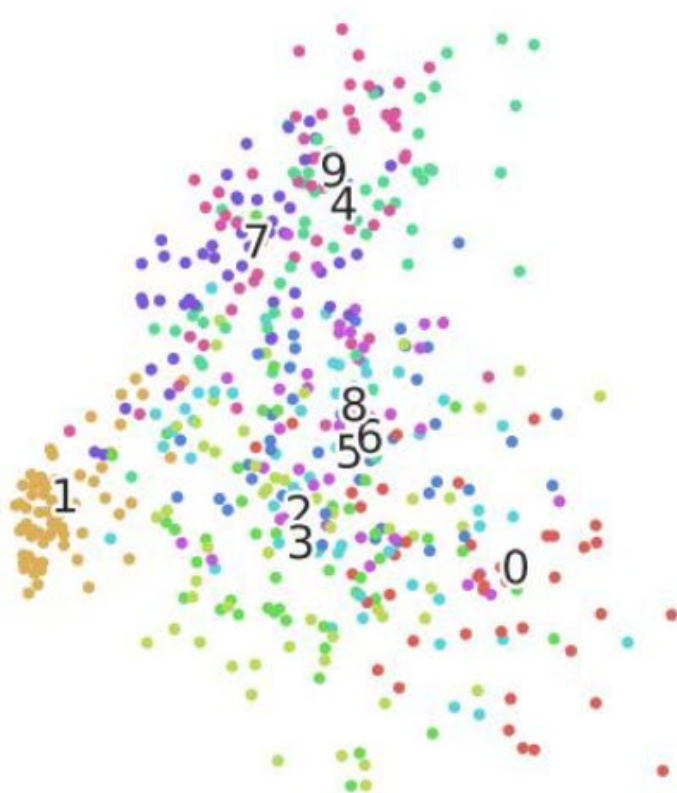
- Linear



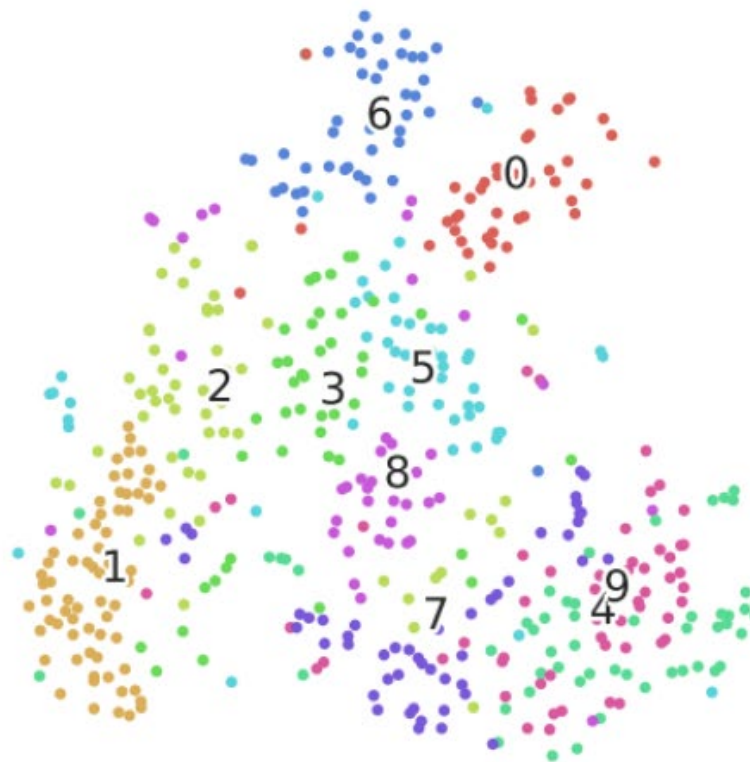
http://www.astroml.org/book_figures/chapter7/fig_S_manifold_PCA.html



Weakness of PCA



Pixel (28x28) -> PCA (2)

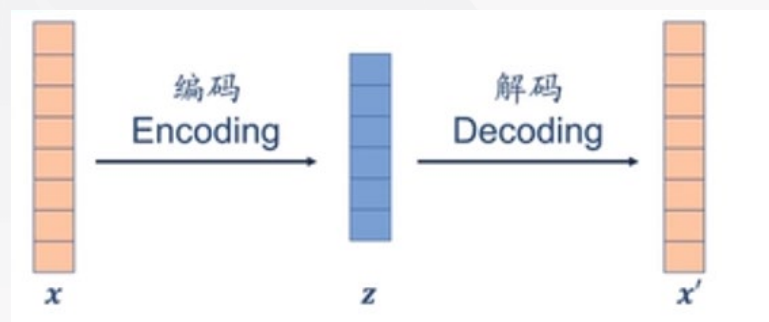


Pixel (28x28) -> tSNE (2)

线性编码



- 给定一组基向量 $A=[a_1, \dots, a_M]$ ，将输入样本 x 表示为这些基向量的线性组合



$$x = \sum_{m=1}^M z_m a_m$$
$$= Az,$$

字典 (dictionary)

编码 (encoding)

$x = Az$

x A z

完备性



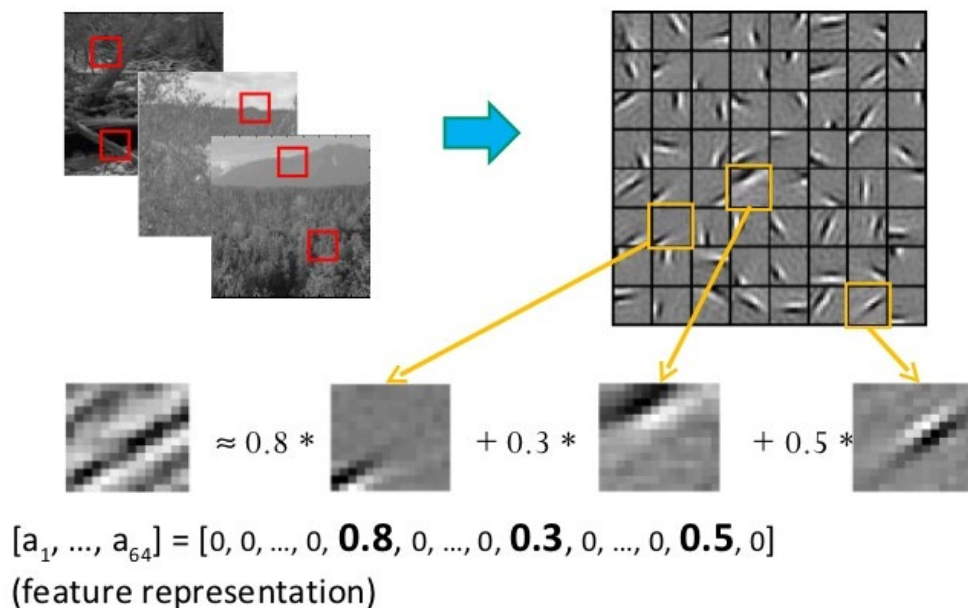
数学小知识 | 完备性

如果 M 个基向量刚好可以支撑 M 维的欧氏空间, 则这 M 个基向量是完备的. 如果 M 个基向量可以支撑 D 维的欧氏空间, 并且 $M > D$, 则这 M 个基向量是过完备的 (overcomplete)、冗余的.

“过完备”基向量是指基向量个数远远大于其支撑空间维度. 因此这些基向量一般不具备独立、正交等性质.

- 稀疏编码
找到一组“过完备”的基向量 (即 $M > D$) 来进行编码。

Sparse coding illustration





- 给定一组 N 个输入向量 $\mathbf{x}^{(1)}, \dots, \mathbf{x}^{(N)}$, 其稀疏编码的目标函数定义为

$$\mathcal{L}(\mathbf{A}, \mathbf{Z}) = \sum_{n=1}^N \left(\left\| \mathbf{x}^{(n)} - \mathbf{A}\mathbf{z}^{(n)} \right\|^2 + \eta \rho(\mathbf{z}^{(n)}) \right)$$

- $\rho(\cdot)$ 是一个稀疏性衡量函数, η 是一个超参数, 用来控制稀疏性的强度。

$$\rho(\mathbf{z}) = \sum_{i=1}^p \mathbf{I}(|z_i| > 0)$$

$$\rho(\mathbf{z}) = \sum_{i=1}^p -\exp(-z_i^2)$$

$$\rho(\mathbf{z}) = \sum_{i=1}^p |z_i|$$

$$\rho(\mathbf{z}) = \sum_{i=1}^p \log(1 + z_i^2)$$

训练过程



- 稀疏编码的训练过程一般用交替优化的方法进行。

1) 固定基向量 $\mathbf{A}$, 对每个输入 $\mathbf{x}^{(n)}$, 计算其对应的最优编码

$$\min_{\mathbf{z}^{(n)}} \left\| \mathbf{x}^{(n)} - \mathbf{A}\mathbf{z}^{(n)} \right\|^2 + \eta \rho(\mathbf{z}^{(n)}), \quad \forall n \in [1, N].$$

2) 固定上一步得到的编码 $\{\mathbf{z}^{(n)}\}_{n=1}^N$, 计算其最优的基向量

$$\min_{\mathbf{A}} \sum_{n=1}^N \left(\left\| \mathbf{x}^{(n)} - \mathbf{A}\mathbf{z}^{(n)} \right\|^2 \right) + \lambda \frac{1}{2} \|\mathbf{A}\|^2,$$

稀疏编码的优点



计算量

稀疏性带来的最大好处就是可以极大地降低计算量。

可解释性

因为稀疏编码只有少数的非零元素，相当于将一个输入样本表示为少数几个相关的特征。这样我们可以更好地描述其特征，并易于理解。

特征选择

稀疏性带来的另外一个好处是可以实现特征的自动选择，只选择和输入样本相关的最少特征，从而可以更好地表示输入样本，降低噪声并减轻过拟合。

自编码器

Auto-Encoder



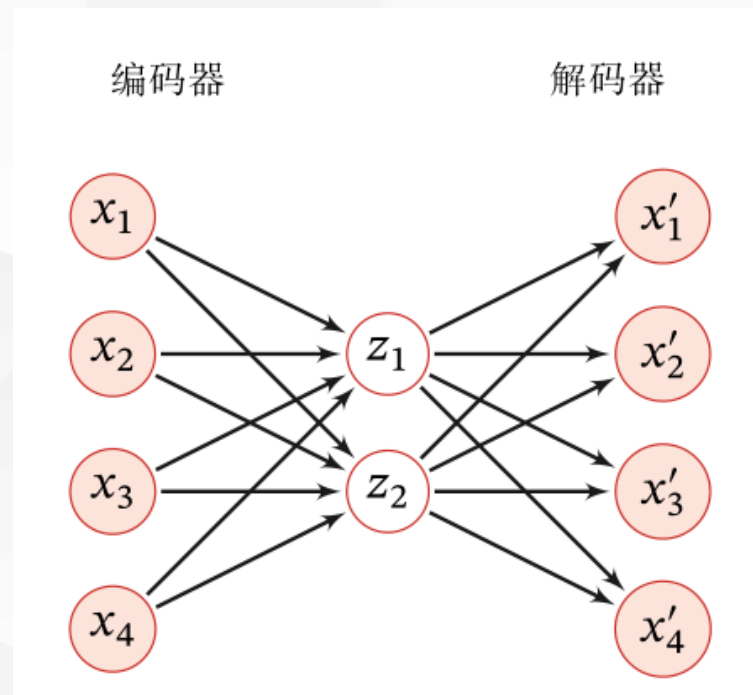
- 编码器 (Encoder)
- 解码器 (Decoder)
- 目标函数：重构错误

$$f : \mathbb{R}^D \rightarrow \mathbb{R}^M$$

$$g : \mathbb{R}^M \rightarrow \mathbb{R}^D$$

$$\begin{aligned}\mathcal{L} &= \sum_{n=1}^N \|\mathbf{x}^{(n)} - g(f(\mathbf{x}^{(n)}))\|^2 \\ &= \sum_{n=1}^N \|\mathbf{x}^{(n)} - f \circ g(\mathbf{x}^{(n)})\|^2.\end{aligned}$$

- 两层网络结构的自编码器



自编码器

Auto-Encoder



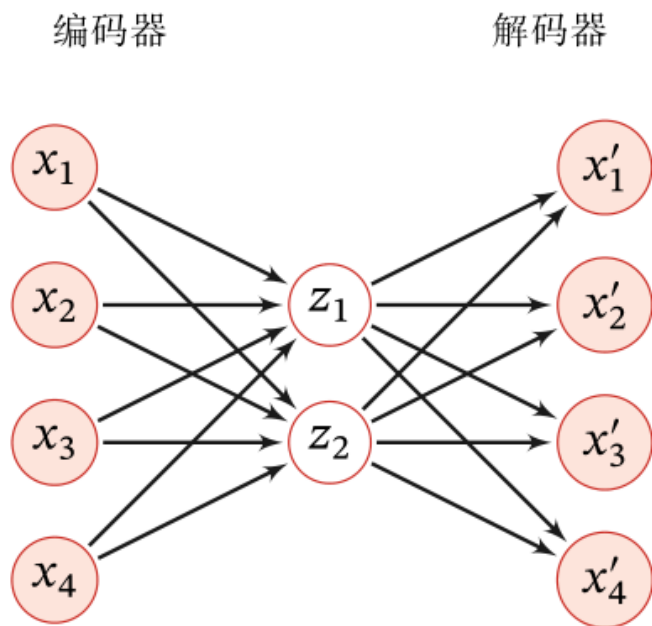
- 编码器 (Encoder) $f : \mathbb{R}^D \rightarrow \mathbb{R}^M$

- 解码器 (Decoder) $g : \mathbb{R}^M \rightarrow \mathbb{R}^D$

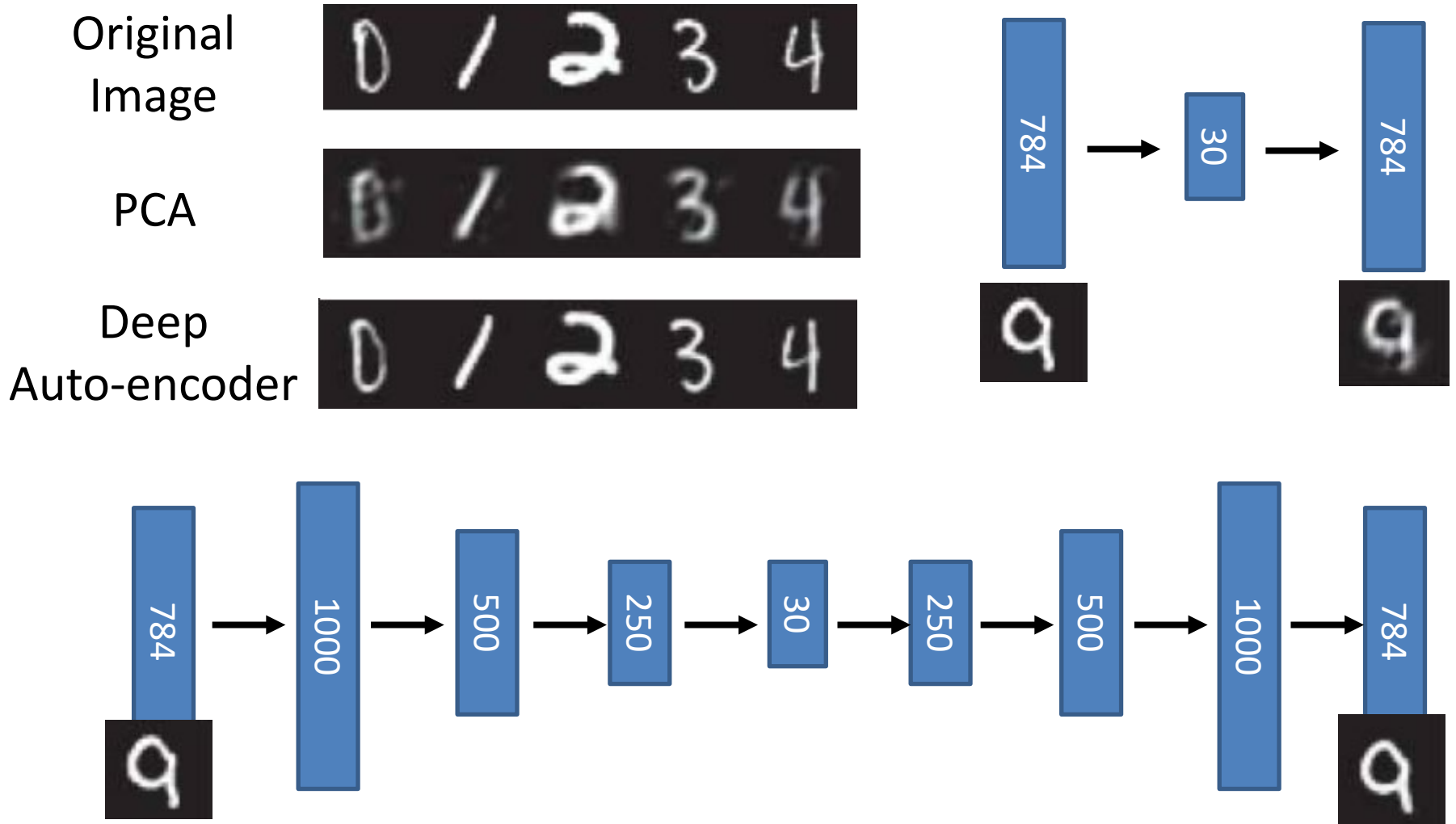
- 目标函数：重构错误

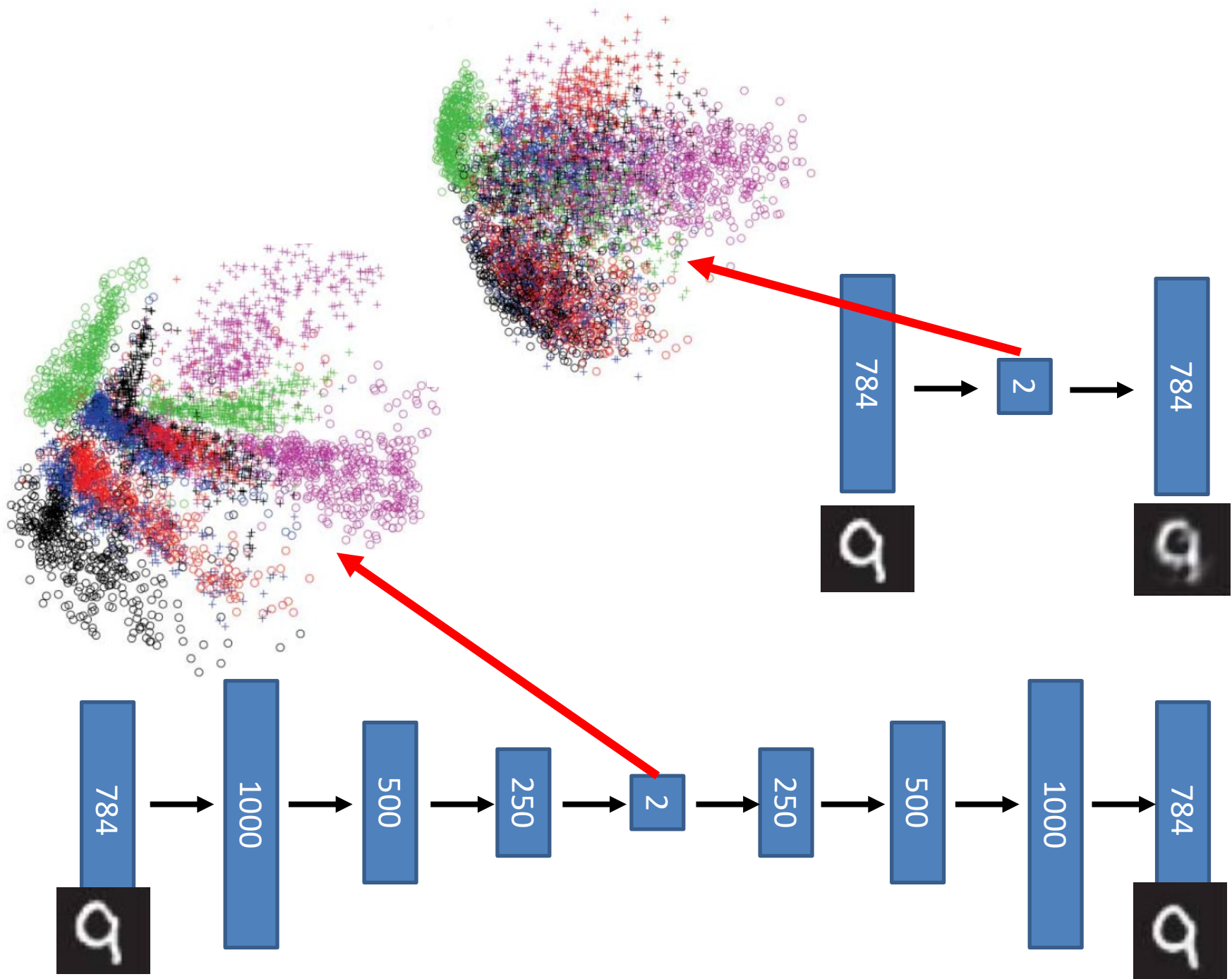
$$\begin{aligned}\mathcal{L} &= \sum_{n=1}^N \|\mathbf{x}^{(n)} - g(f(\mathbf{x}^{(n)}))\|^2 \\ &= \sum_{n=1}^N \|\mathbf{x}^{(n)} - f \circ g(\mathbf{x}^{(n)})\|^2.\end{aligned}$$

- 两层网络结构的自编码器



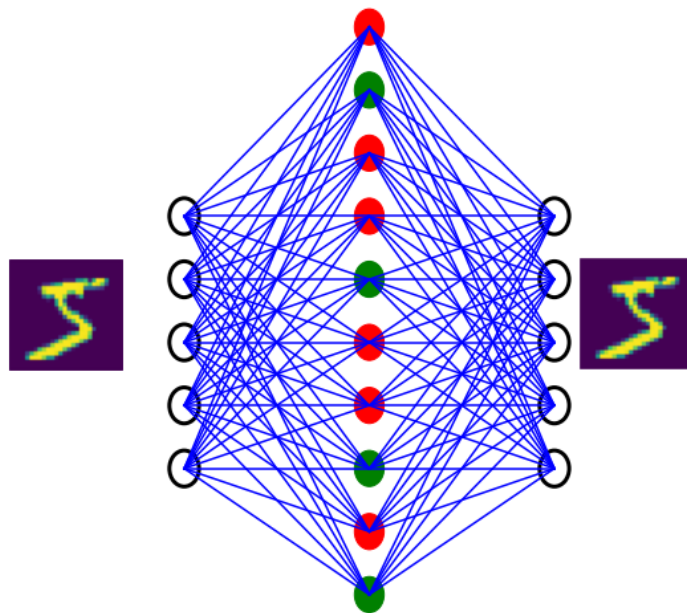
Deep Auto-encoder





- $$\mathcal{L} = \sum_{n=1}^N \|\mathbf{x}^{(n)} - \mathbf{x}'^{(n)}\|^2 + \eta \rho(\mathbf{Z}) + \lambda \|\mathbf{W}\|^2$$

和稀疏编码一样，稀疏自编码器的优点是有很高的可解释性，并同时进行了隐式的特征选择。

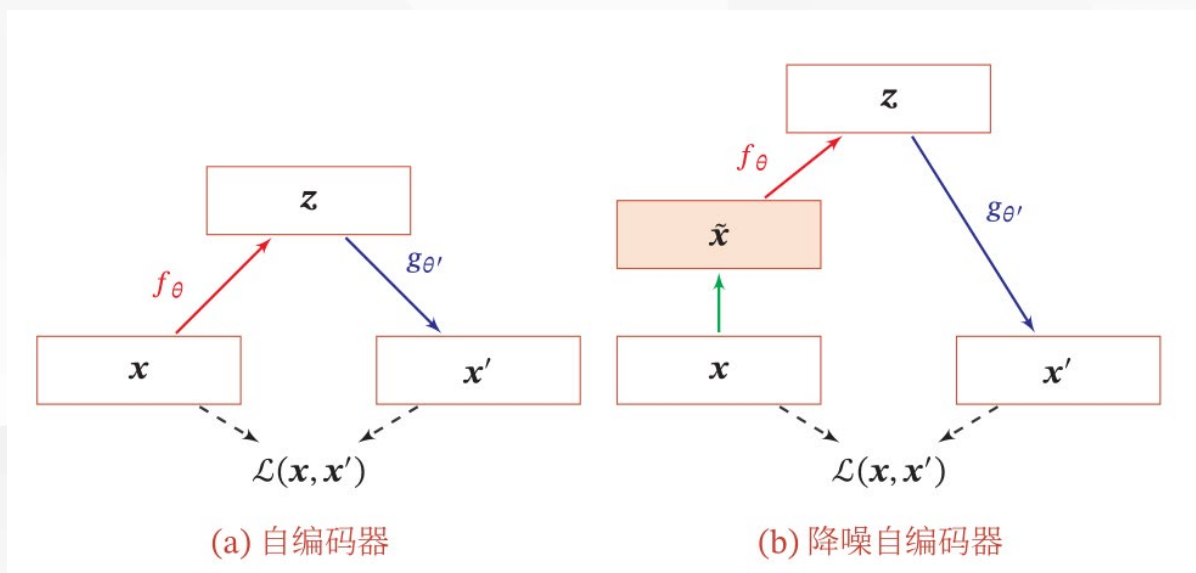


降噪自编码器



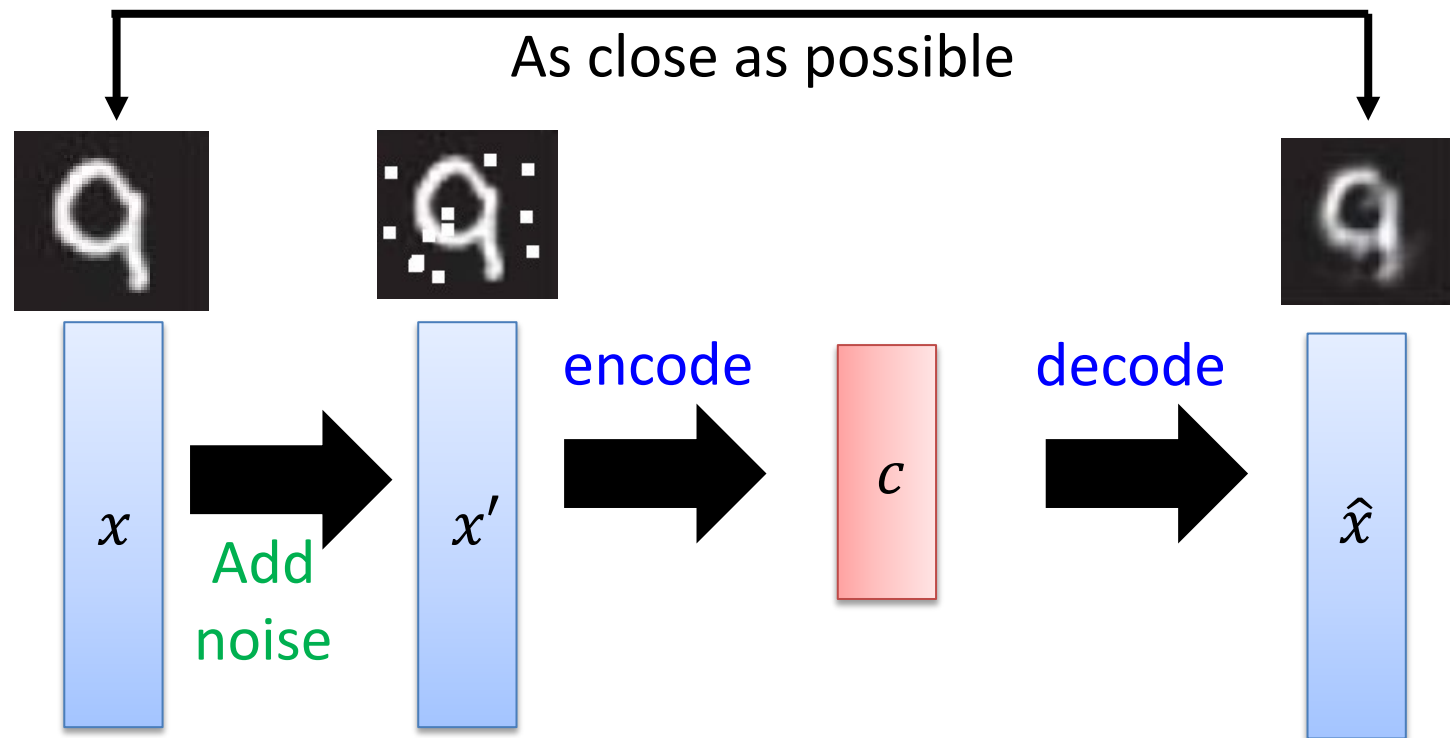
通过引入噪声来增加编码鲁棒性的自编码器

- 对于一个向量 x ，我们首先根据一个比例 μ 随即将 x 的一些维度的值设置为0，得到一个被损坏的向量 $\tilde{x}$ 。
- 然后将被损坏的向量 $\tilde{x}$ 输入给自编码器得到的编码 z ，并重构出原始的无损输入 x 。



Auto-encoder

- De-noising auto-encoder



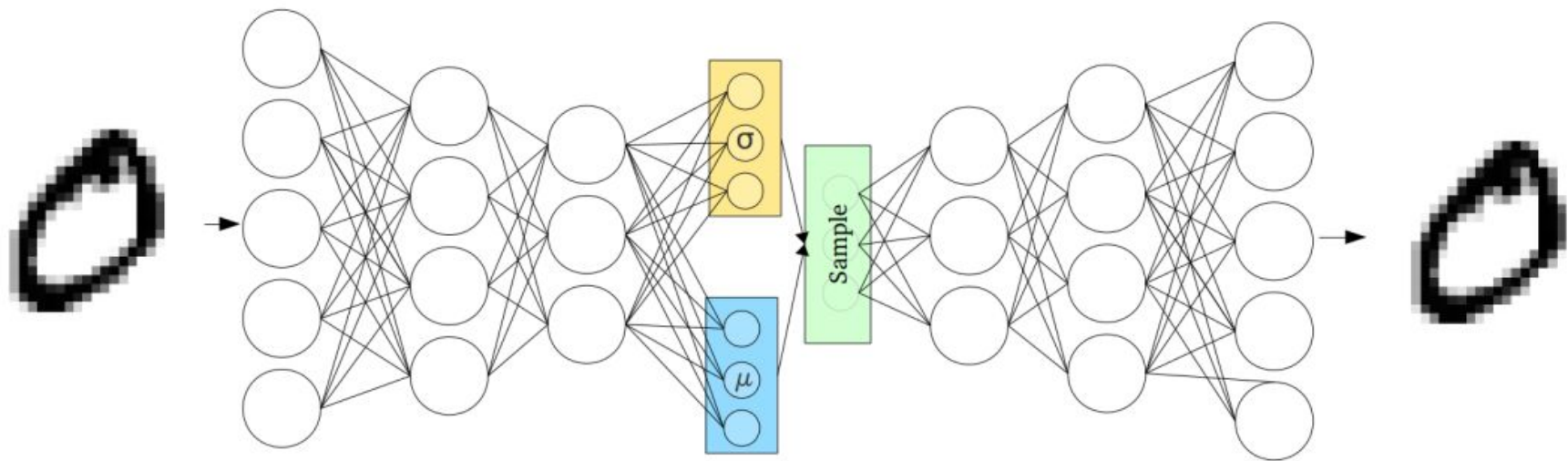
Vincent, Pascal, et al. "Extracting and composing robust features with denoising autoencoders." *ICML*, 2008.

变分自编码器



变分自动编码器（Variational Autoencoder, VAE）是一种基于神经网络的生成模型，其主要目的是学习数据的概率分布并能够生成新的数据样本。VAE最初由Diederik P. Kingma和Max Welling在2013年提出，是对传统自动编码器（Autoencoder, AE）的改进。

VAE引入了一种基于概率的生成模型来描述输入数据的潜在分布。VAE将数据编码为一个潜在变量向量，并使用一个解码器将该向量映射回原始数据空间中。不同于传统自动编码器使用确定性的编码器和解码器，VAE使用随机的编码器将输入数据编码为潜在变量的分布，并通过采样来生成新的数据样本。

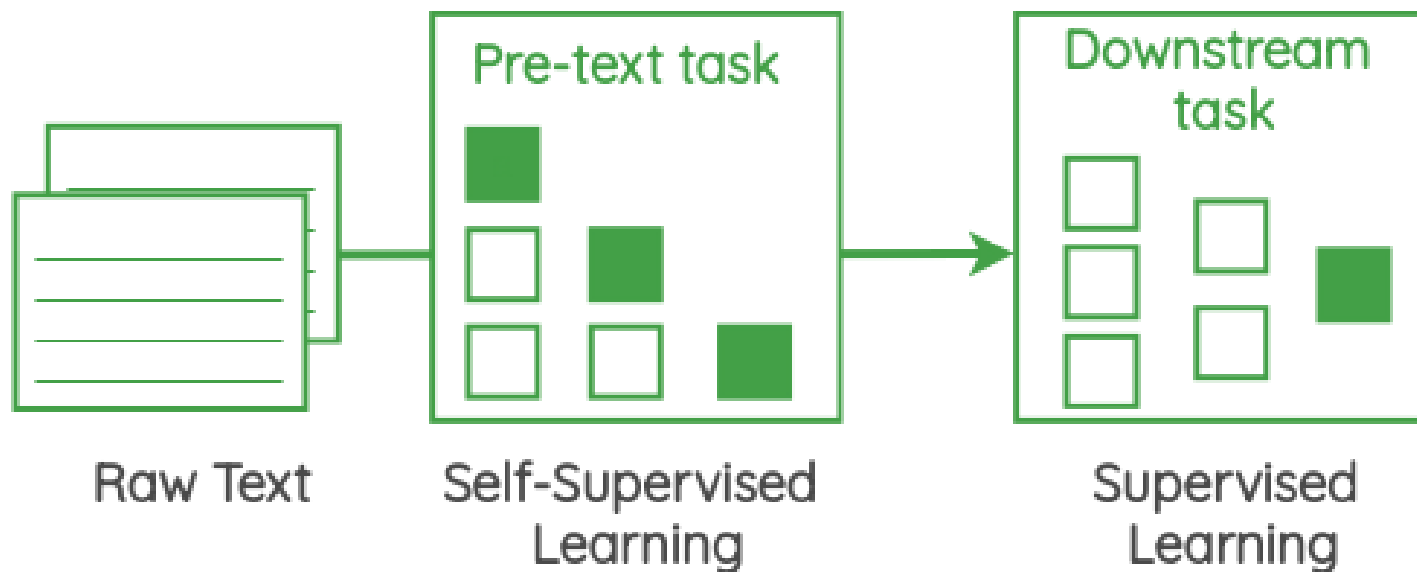


自监督学习



旨在对于**无标签数据**，通过设计**辅助任务（Proxy tasks）**来挖掘数据自身的表征特性作为监督信息，来提升模型的特征提取能力。

1、自然语言处理（NLP）



自监督学习



a、单词预测 (Word prediction)

通过随机删去训练集句子中的单词来构造辅助任务训练集和标签，来训练网络预测被删去的单词，以提升模型对于语序特征的提取能力。

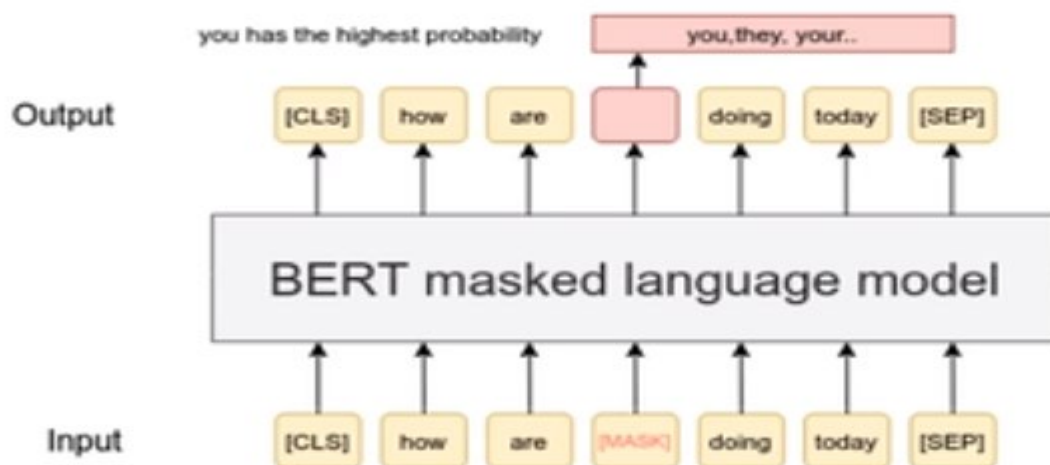


Fig. 1. BERT-Original sentence "how are you doing today"



自监督学习



b、句子序列预测（Sentence sequence prediction）

通过随机打乱每段话中的句子顺序来构造辅助任务训练集，来训练网络对句子进行正确的排序，标签为原来正确的句子顺序。

Document

I completed high school. Then I did my undergrad.

...

Sentence 1	Sentence 2	Correct order
I completed high school	Then I did my undergrad	yes
Then I did my undergrad	I completed high school	no

c、词序列预测（Word sequence prediction）

I did X. Then I did Y. Finally I did Z.

自监督学习



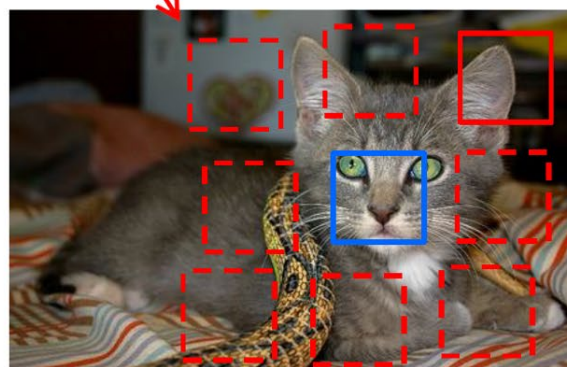
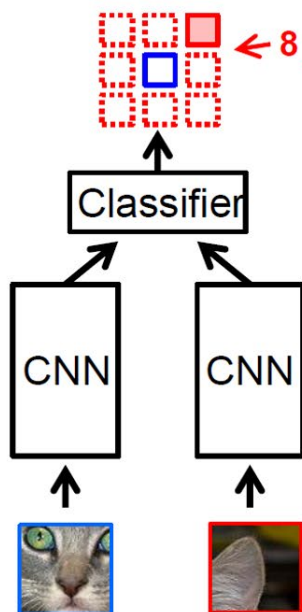
2、计算机视觉（CV）

我们将计算机视觉任务根据输入信息的不同分为图像任务和视频处理任务，来分别介绍其自监督学习中的辅助任务设计。

a、图像重组（Jigsaw Puzzles）

将图像分为不同的patch，比如九宫格，然后让网络预测不同patch的相对位置信息。这个过程中可以提高模型的局部特征提取能力以及全局空间信息提取能力。

Train network to predict relative position of two regions in the same image



Randomly Sample Patch
Sample Second Patch

Unsupervised visual representation learning by context prediction,
Carl Doersch, Abhinav Gupta, Alexei A. Efros, ICCV 2015

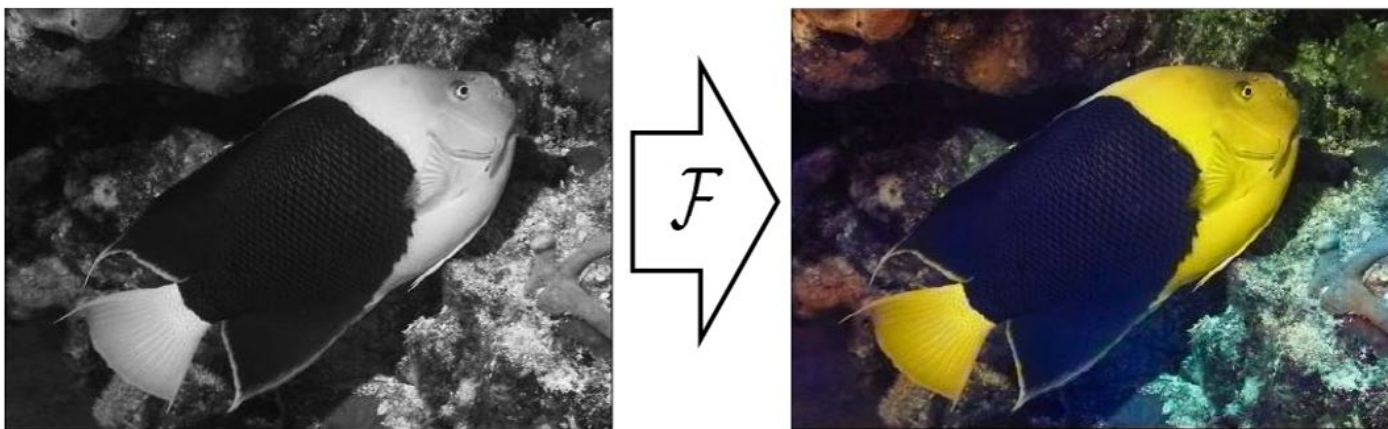
自监督学习



b、图像渲染 (Image Colorization)

将原来数据集中的RGB图像进行灰度化处理，然后通过图像色彩恢复任务来训练网络。

Train network to predict pixel colour from a monochrome input

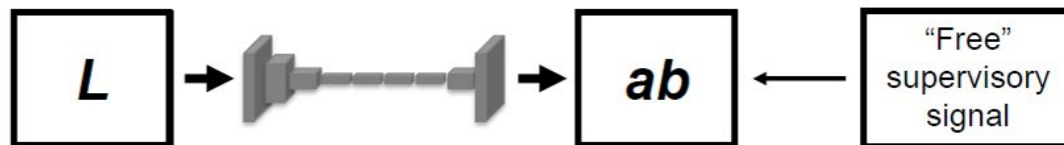


Grayscale image: L channel

$$\mathbf{X} \in \mathbb{R}^{H \times W \times 1}$$

Concatenate (L, ab)

$$(\mathbf{X}, \hat{\mathbf{Y}})$$

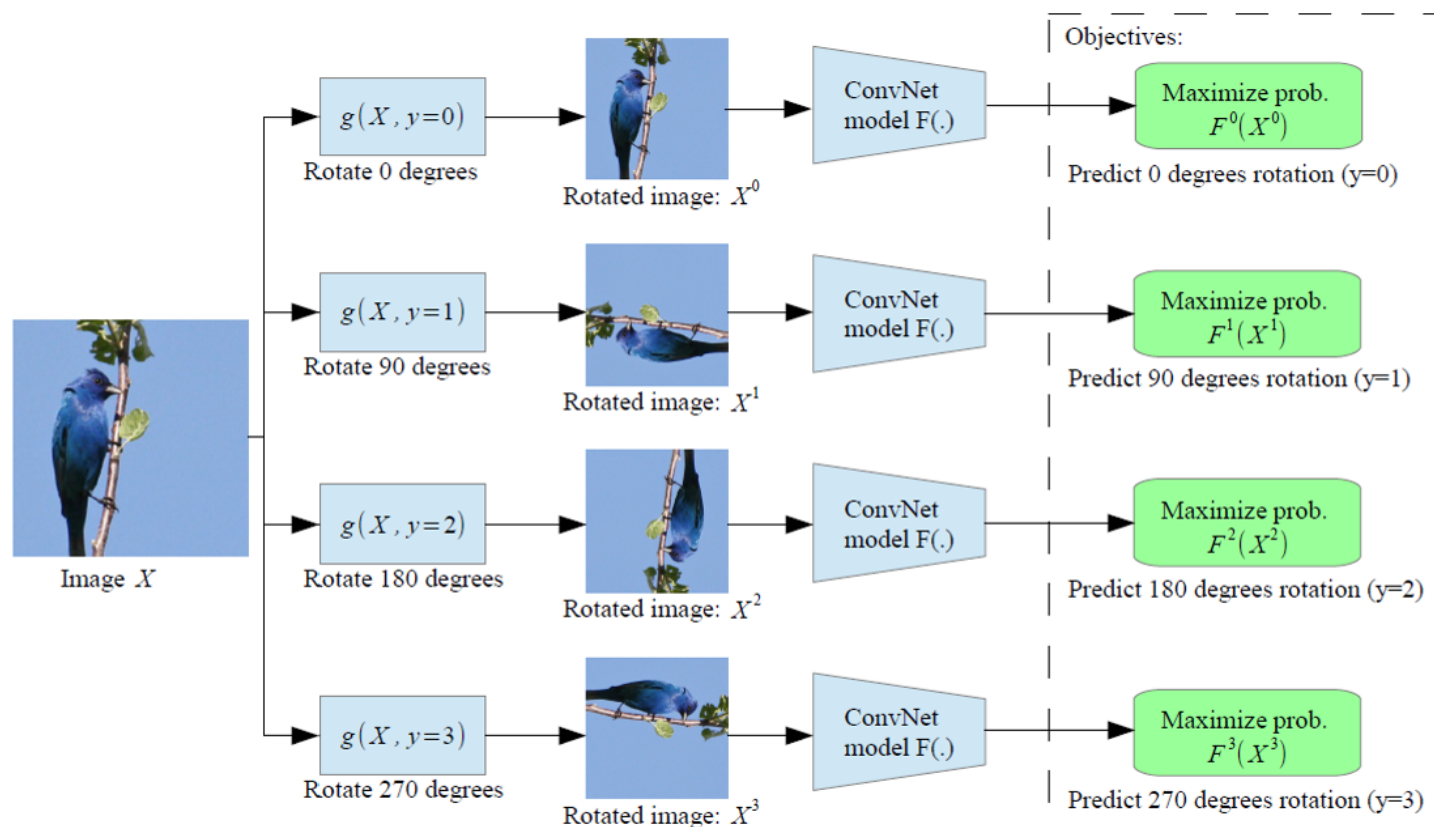


自监督学习



c、图像旋转角度预测（Image Colorization）

将训练集中的图像进行随机旋转，然后通过旋转角回归任务来训练网络。



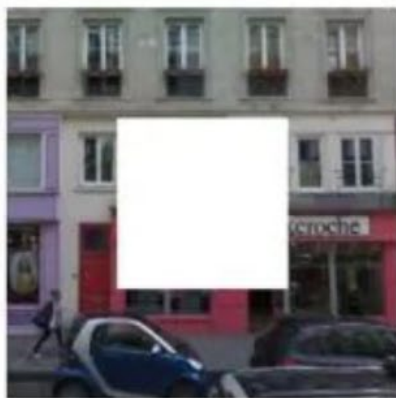
Unsupervised representation learning by predicting image rotations,
Spyros Gidaris, Praveer Singh, Nikos Komodakis, ICLR 2018

自监督学习

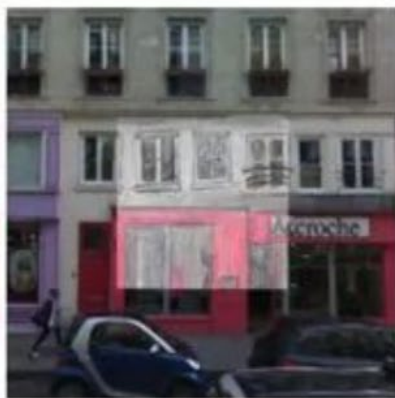


d、图像修复 (Image In-painting)

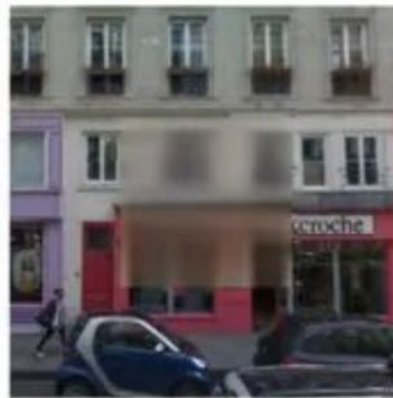
对图像进行随机裁剪，训练网络修复图像。



(a) Input context



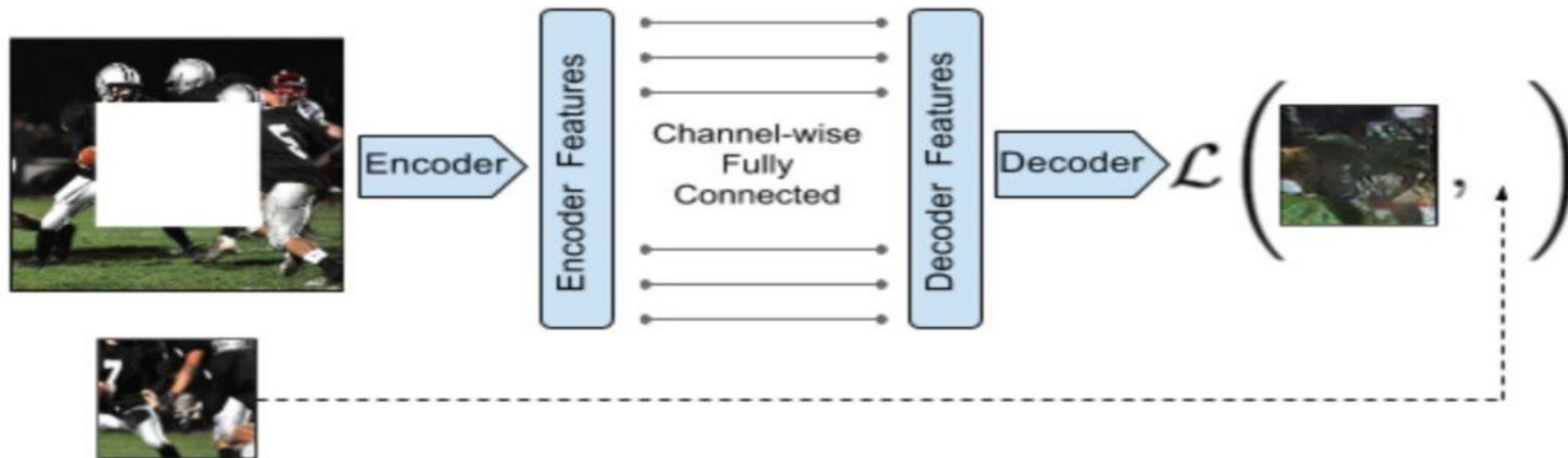
(b) Human artist



(c) Context Encoder
(L_2 loss)



(d) Context Encoder

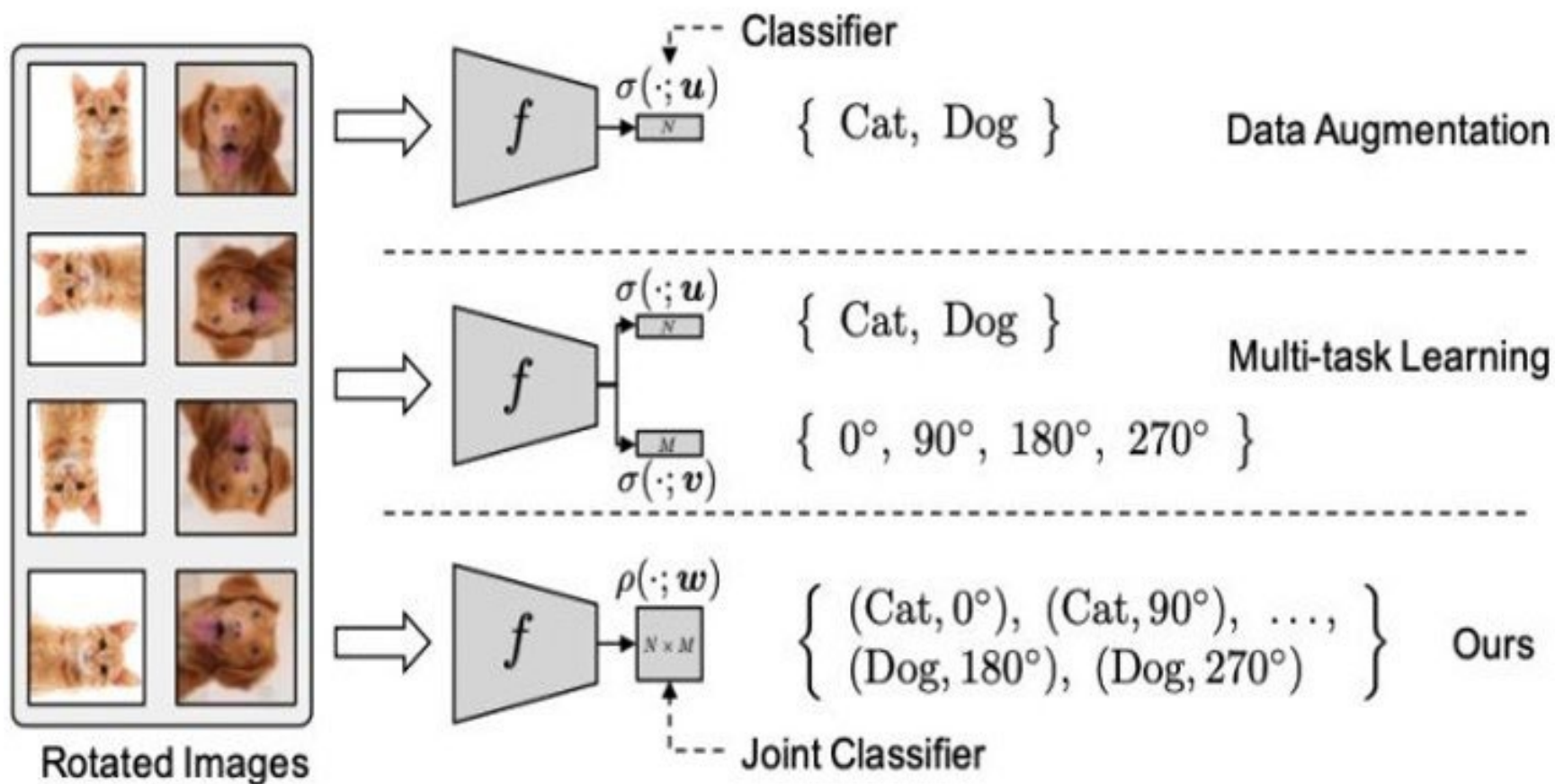


自监督学习



e、多任务学习 (Multi-Tasks)

结合上述的几种辅助任务一起对模型进行训练。



自监督学习



3、视频处理任务

视频是一系列图像帧的时间序列，时序相邻帧之间具有很强的关联信息，较远帧之间的关联较弱，所以视频帧的时序信息是我们可以用到的一个重要设计依据；此外，视频中的目标物体在不同时序帧中具有一致性，比如色彩、形状等，所以这也可以作为一个辅助任务设计的重要依据。

a、基于视频帧的序列信息

与自然语言处理中的语序预测很相似，我们通过随机打乱训练集中视频帧的顺序，来训练网络让其对正确视频时序进行预测。

Learning the arrow of time

Task: predict if video playing forwards or backwards

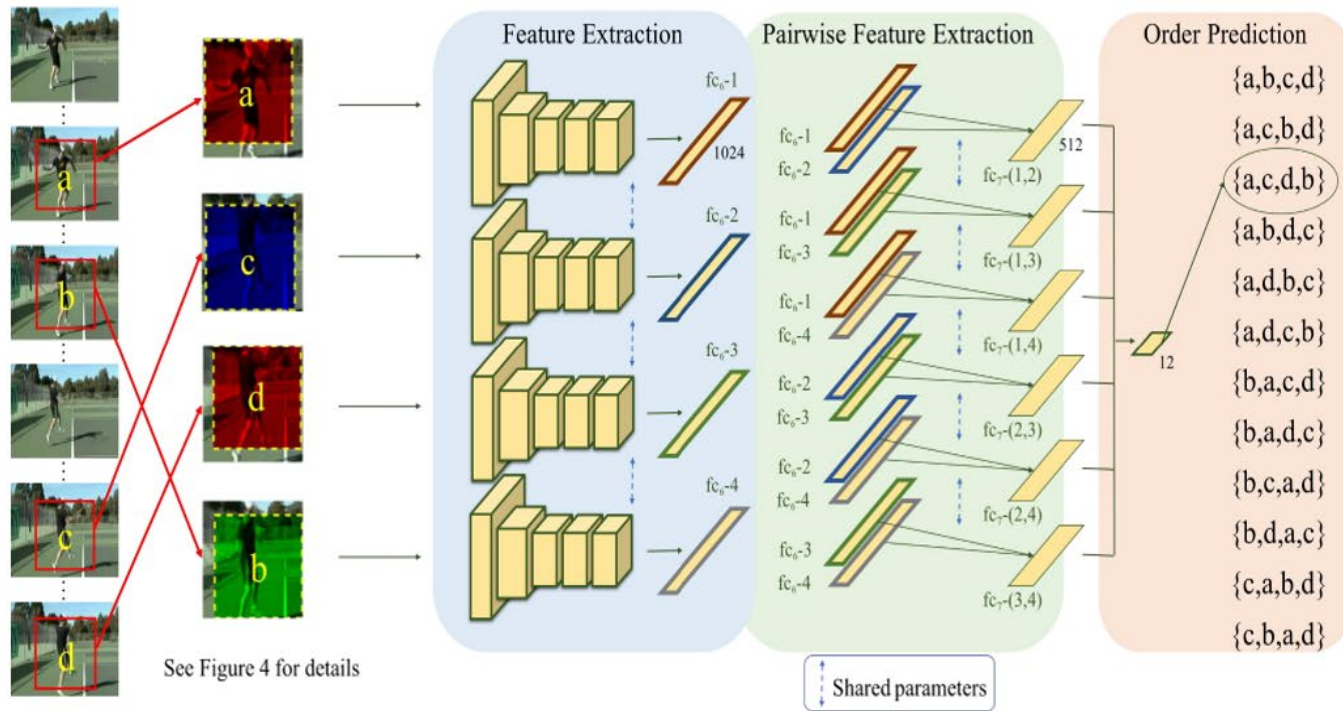


Supervision:

Positive training samples: video clips playing forwards

Negative training samples: video clips playing backwards

自监督学习



Harder task than
Shuffle and Learn:

12 way classification
(here) vs binary for
Shuffle and Learn

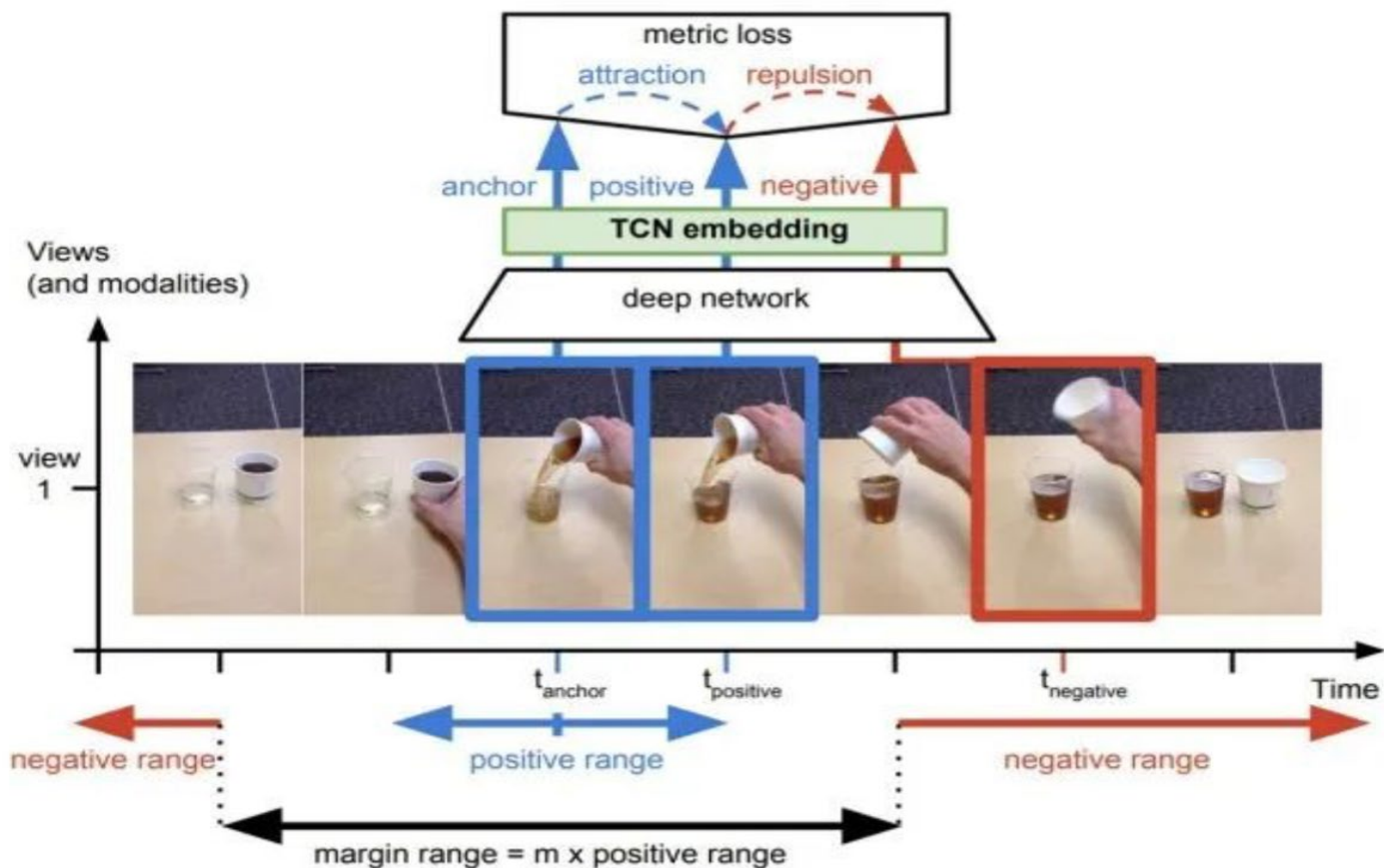
Unsupervised representation learning by sorting sequence,
H. Lee, J. Huang, M. K. Singh, and M. Yang, ICCV 2017

自监督学习



b、基于视频中目标的相似性

一般来说，视频相邻帧中目标相似性比较高，相隔较远帧中目标相似性比较低，所以可以训练网络判断不同帧中目标的相似性来提升其特征提取能力。

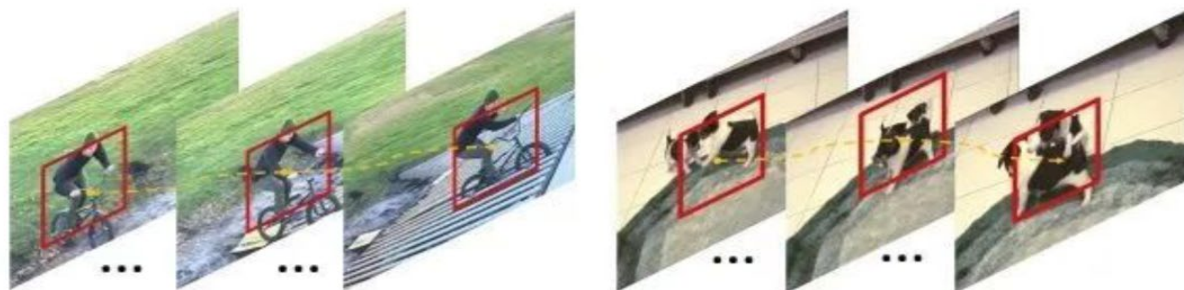


自监督学习

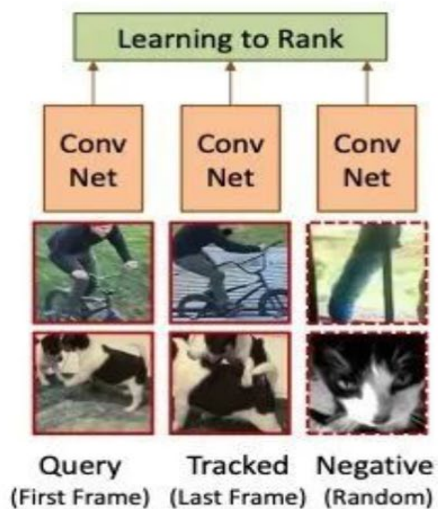


c、基于无监督目标跟踪

视频中同一物体在不同帧的特征应该是相似的，而不同物体的特征是不同的。可以根据无监督模型获得目标跟踪框，然后让网络学习同一目标和不同目标在不同帧中的相似性判别来提升特征提取能力。



(a) Unsupervised Tracking in Videos



(b) Siamese-triplet Network

$$\begin{aligned} D \left(\begin{bmatrix} \text{Query} & \text{Tracked} \end{bmatrix} \right) &< D \left(\begin{bmatrix} \text{Query} & \text{Negative} \end{bmatrix} \right) \\ D \left(\begin{bmatrix} \text{Query} & \text{Tracked} \end{bmatrix} \right) &< D \left(\begin{bmatrix} \text{Query} & \text{Negative} \end{bmatrix} \right) \end{aligned}$$

D : Distance in deep feature space

(c) Ranking Objective

概率密度估计



参数密度估计

(Parametric Density Estimation)

- 根据先验知识假设随机变量服从某种分布，然后通过训练样本来估计分布的参数
- 估计方法：最大似然估计

$$\log p(\mathcal{D}; \theta) = \sum_{n=1}^N \log p(\mathbf{x}^{(n)}; \theta).$$

非参数密度估计

(Nonparametric Density Estimation)

- 不假设数据服从某种分布，通过将样本空间划分为不同的区域并估计每个区域的概率来近似数据的概率密度函数。

参数密度估计



- 正态分布

假设样本 $\mathbf{x} \in \mathbb{R}^D$ 服从正态分布

$$\mathcal{N}(\mathbf{x}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = \frac{1}{(2\pi)^{D/2} |\boldsymbol{\Sigma}|^{1/2}} \exp\left(-\frac{1}{2}(\mathbf{x} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x} - \boldsymbol{\mu})\right),$$

其中 $\boldsymbol{\mu}$ 和 $\boldsymbol{\Sigma}$ 分别为正态分布的均值和方差.

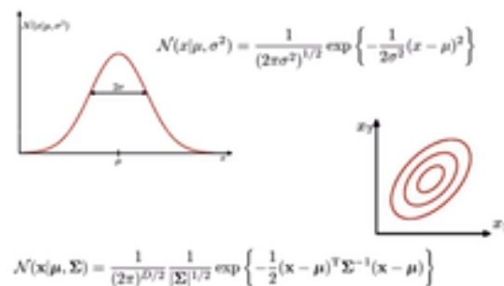
数据集 $\mathcal{D} = \{\mathbf{x}^{(n)}\}_{n=1}^N$ 的对数似然函数为

$$\log p(\mathcal{D}|\boldsymbol{\mu}, \boldsymbol{\Sigma}) = -\frac{N}{2} \log((2\pi)^D |\boldsymbol{\Sigma}|) - \frac{1}{2} \sum_{n=1}^N (\mathbf{x}^{(n)} - \boldsymbol{\mu})^\top \boldsymbol{\Sigma}^{-1}(\mathbf{x}^{(n)} - \boldsymbol{\mu}).$$

分别求上式关于 $\boldsymbol{\mu}, \boldsymbol{\Sigma}$ 的偏导数, 并令其等于 0. 可得,

$$\boldsymbol{\mu}^{ML} = \frac{1}{N} \sum_{n=1}^N \mathbf{x}^{(n)},$$

$$\boldsymbol{\Sigma}^{ML} = \frac{1}{N} \sum_{n=1}^N (\mathbf{x}^{(n)} - \boldsymbol{\mu}^{ML})(\mathbf{x}^{(n)} - \boldsymbol{\mu}^{ML})^\top.$$



参数密度估计



- 多项分布

数据集 $\mathcal{D} = \{\mathbf{x}^{(n)}\}_{n=1}^N$ 的对数似然函数为

$$\log p(\mathcal{D}|\boldsymbol{\mu}) = \sum_{n=1}^N \sum_{k=1}^K x_k^{(n)} \log(\mu_k). \quad (9.34)$$

多项分布的参数估计为约束优化问题. 引入拉格朗日乘子 λ , 将原问题转换为无约束优化问题.

$$\max_{\boldsymbol{\mu}, \lambda} \sum_{n=1}^N \sum_{k=1}^K x_k^{(n)} \log(\mu_k) + \lambda \left(\sum_{k=1}^K \mu_k - 1 \right). \quad (9.35)$$

分别求上式关于 μ_k, λ 的偏导数, 并令其等于 0. 可得,

$$\mu_k^{ML} = \frac{m_k}{N}, \quad 1 \leq k \leq K \quad (9.36)$$

其中 $m_k = \sum_{n=1}^N x_k^{(n)}$ 为数据集中取值为第 k 个状态的样本数量.

参数密度估计一般存在以下问题



模型选择问题

- 如何选择数据分布的密度函数？
- 实际数据的分布往往是非常复杂的，而不是简单的正态分布或多项分布。

不可观测变量问题

- 即我们用来训练的样本只包含部分的可观测变量，还有一些非常关键的变量是无法观测的，这导致我们很难准确估计数据的真实分布。

维度灾难问题

- 高维数据的参数估计十分困难
- 随着维度的增加，估计参数所需要的样本数量指数增加。在样本不足时会出现过拟合。

非参数密度估计



- 对于高维空间中的一个随机向量 $\mathbf{x}$ ，假设其服从一个未知分布 $p(\mathbf{x})$ ，则 $\mathbf{x}$ 落入空间中的小区域 R 的概率为

$$P = \int_{\mathcal{R}} p(\mathbf{x}) d\mathbf{x}.$$

- 给定 N 个训练样本 $\mathbf{D} = \{\mathbf{x}^{(n)}\}_{n=1}^N$ ，落入区域 R 的样本数量 K 服从二项分布

$$P_K = \binom{N}{K} P^K (1 - P)^{1-K},$$

- 当 N 非常大时，我们可以近似认为 $P \approx \frac{K}{N}$
- 假设区域 R 足够小，其内部的概率密度是相同的，则有 $P \approx p(\mathbf{x})V$
- 结合上述两个公式，得到 $p(\mathbf{x}) \approx \frac{K}{NV}$

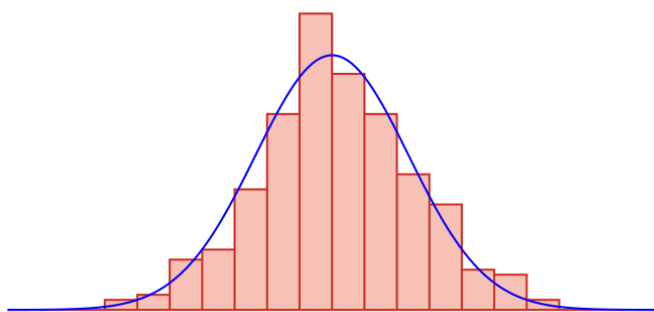
直方图方法

Histogram Method

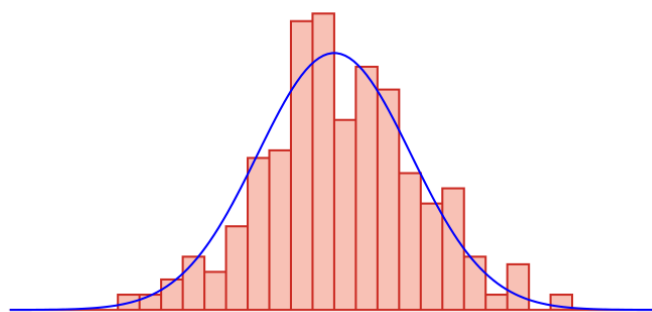


- 一种非常直观的估计连续变量密度函数的方法，可以表示为一种柱状图。

以一维随机变量为例，首先将其取值范围分成 M 个连续的、不重叠的区间 (bin), 每个区间的宽度为 Δ_m . 给定 N 个训练样本 $\mathcal{D} = \{x^{(n)}\}_{n=1}^N$, 我们统计这些样本落入每个区间的数量 K_m , 然后将它们归一化为密度函数.



(a) 10 个区间 (bin)



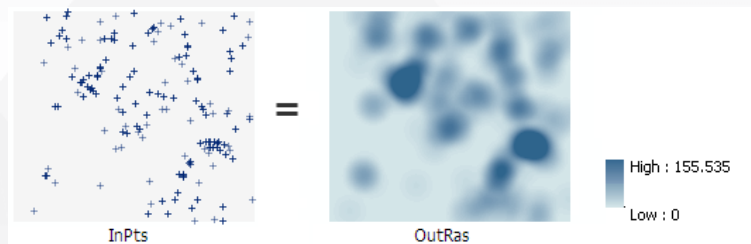
(b) 30 个区间 (bin)

核密度估计

Kernel Density Estimation



- 核密度估计是一种直方图方法的改进，也叫Parzen窗方法。



- 假设 R 为 D 维空间中的一个以点 $\mathbf{x}$ 为中心的“超立方体”，并定义核函数来表示一个样本 $\mathbf{z}$ 是否落入该超立方体中

$$\phi\left(\frac{\mathbf{z} - \mathbf{x}}{H}\right) = \begin{cases} 1 & \text{if } |z_i - x_i| < \frac{H}{2}, 1 \leq i \leq D \\ 0 & \text{else} \end{cases}$$

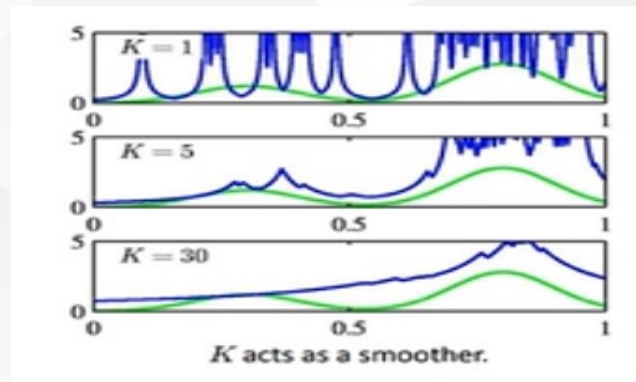
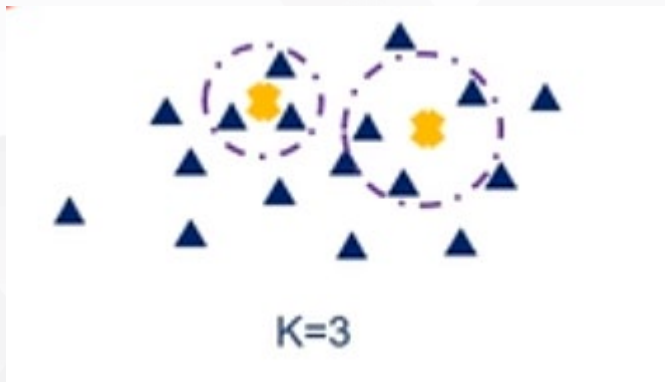
- 点 $\mathbf{x}$ 的密度估计为

$$p(\mathbf{x}) = \frac{K}{NH^D} = \frac{1}{NH^D} \sum_{n=1}^N \phi\left(\frac{\mathbf{x}^{(n)} - \mathbf{x}}{H}\right)$$

K近邻方法



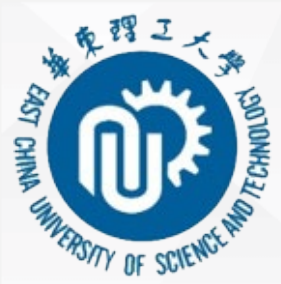
- 核密度估计方法中的核宽度是固定的，因此同一个宽度可能对高密度的区域过大，而对低密度区域过小。
- 一种更灵活的方式是设置一种可变宽度的区域，并使得落入每个区域中样本数量为固定的 K 。
- 要估计点 $\mathbf{x}$ 的密度，首先找到一个以 $\mathbf{x}$ 为中心的球体，使得落入球体的样本数量为 K ，就可以计算出点 $\mathbf{x}$ 的密度。



非参数密度估计



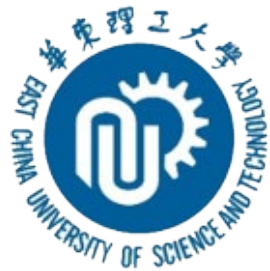
- 非参数密度估计(直方图方法除外)需要保留整个训练集。
- 而参数密度估计不需要保留整个训练集，因此在存储和计算上更加高效。



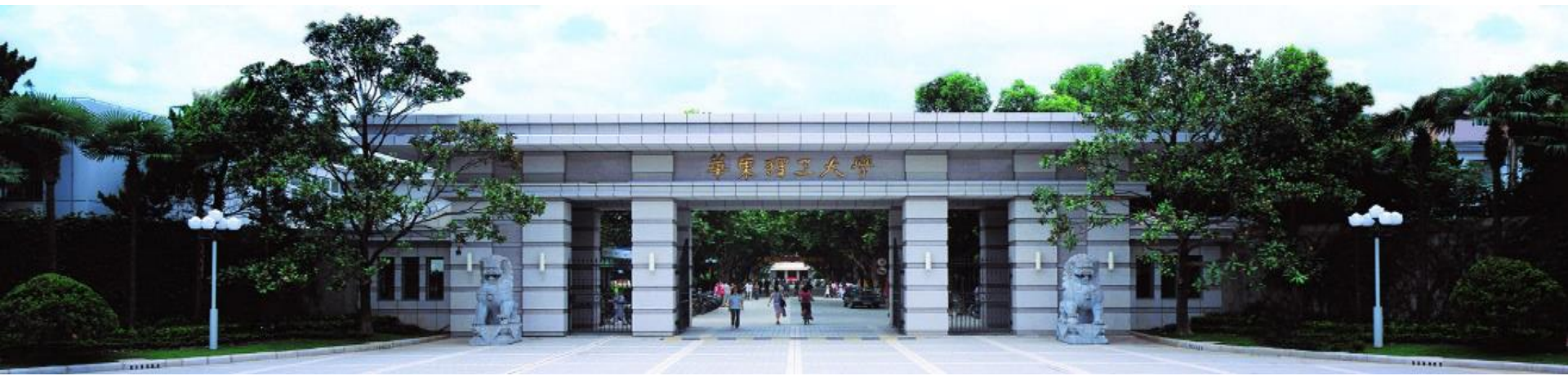
谢谢！

THANK YOU FOR LISTENING





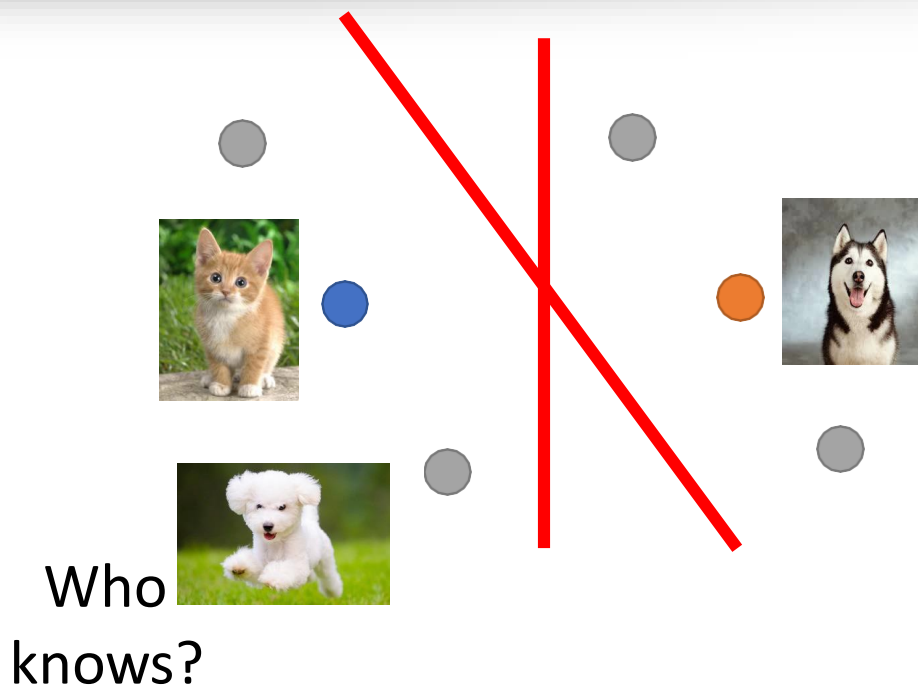
半监督学习（了解）





- Supervised learning: $\{(x^r, \hat{y}^r)\}_{r=1}^R$
 - E.g. x^r : image, $\hat{y}^r$: class labels
- Semi-supervised learning: $\{(x^r, \hat{y}^r)\}_{r=1}^R, \{x^u\}_{u=R}^{R+U}$
 - A set of unlabeled data, usually $U \gg R$
 - Transductive learning: unlabeled data is the testing data
 - Inductive learning: unlabeled data is not the testing data
- Why semi-supervised learning?
 - Collecting data is easy, but collecting “labelled” data is expensive
 - We do semi-supervised learning in our lives

简介



The distribution of the unlabeled data tell us ***something***.

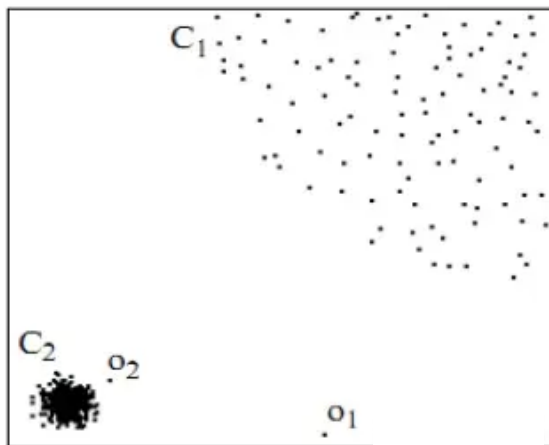
Usually with some assumptions

三大基本假设



(1) 平滑假设(Smoothness Assumption):

位于稠密数据区域的两个距离很近的样例的类标签相似，也就是说，当两个样例被稠密数据区域中的边连接时，它们在很大的概率下有相同的类标签；相反地，当两个样例被稀疏数据区域分开时，它们的类标签趋于不同。具体的含义可见下图：



(2) 聚类假设 (Cluster assumption) :

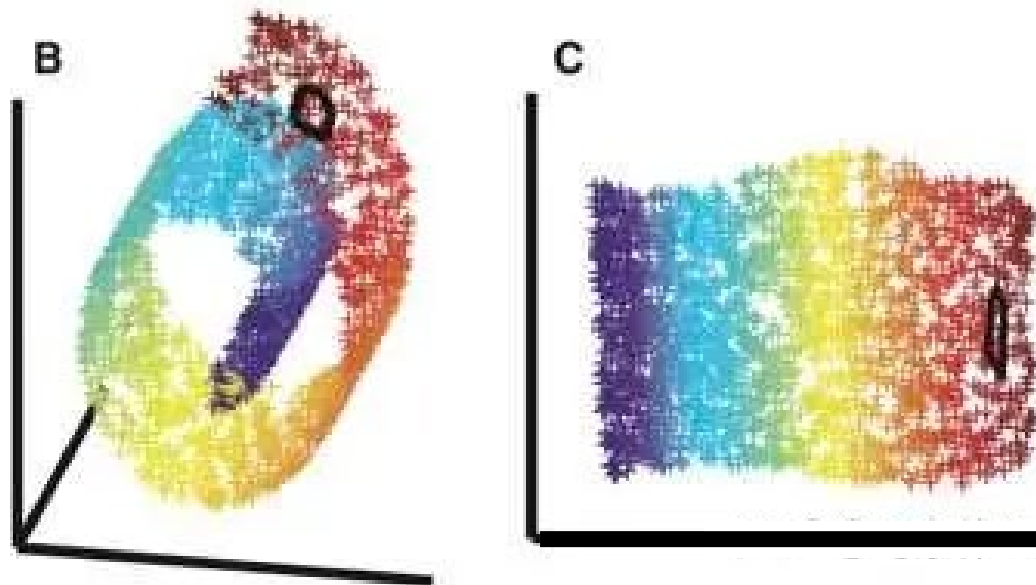
如果样本点在同一簇中，它们很可能属于同一类。实际上聚类假设属于平滑假设的一个特例，在平滑假设中，样本不一定要形成明显的簇，例如整个样本空间都是稠密且均匀的，此时就没有了聚类的簇的概念，当样本空间中形成明显的簇时，聚类假设则派上用场；

三大基本假设



(3) 流形假设 (Manifold assumption) :

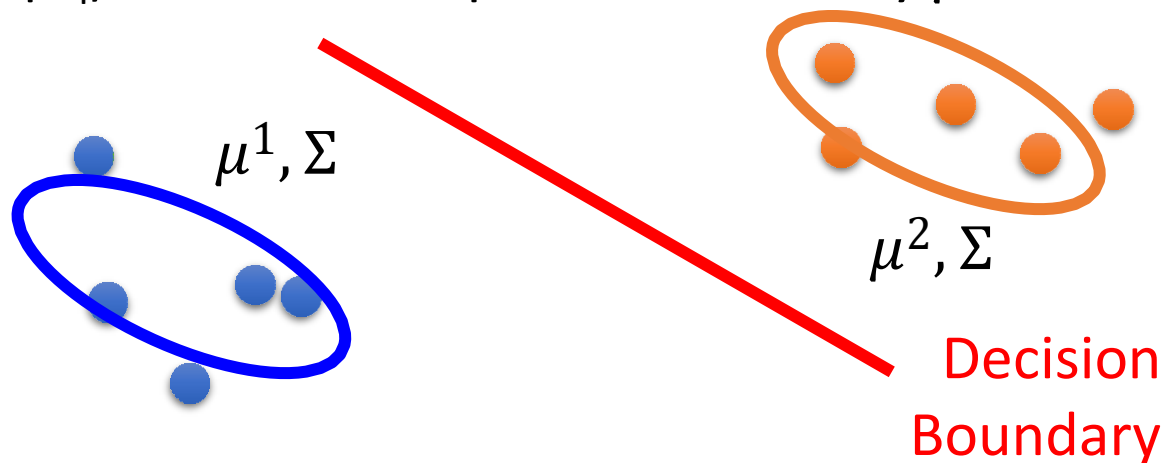
首先我们需要知道当特征的维度非常高的时候会出现维度诅咒的问题，即，在高维空间，距离度量的计算失效，此时密度和距离的概念趋于无效，则上述两大假设难以成立，而流形假设主要内容即假定如果高维样本恰好可以映射到一个低维的流形结构上，此时在低维的流形空间中，前两大假设仍旧是可以成立的。



生成模型



- Given labelled training examples $x^r \in C_1, C_2$
 - looking for most likely prior probability $P(C_i)$ and class-dependent probability $P(x|C_i)$
 - $P(x|C_i)$ is a Gaussian parameterized by μ^i and Σ



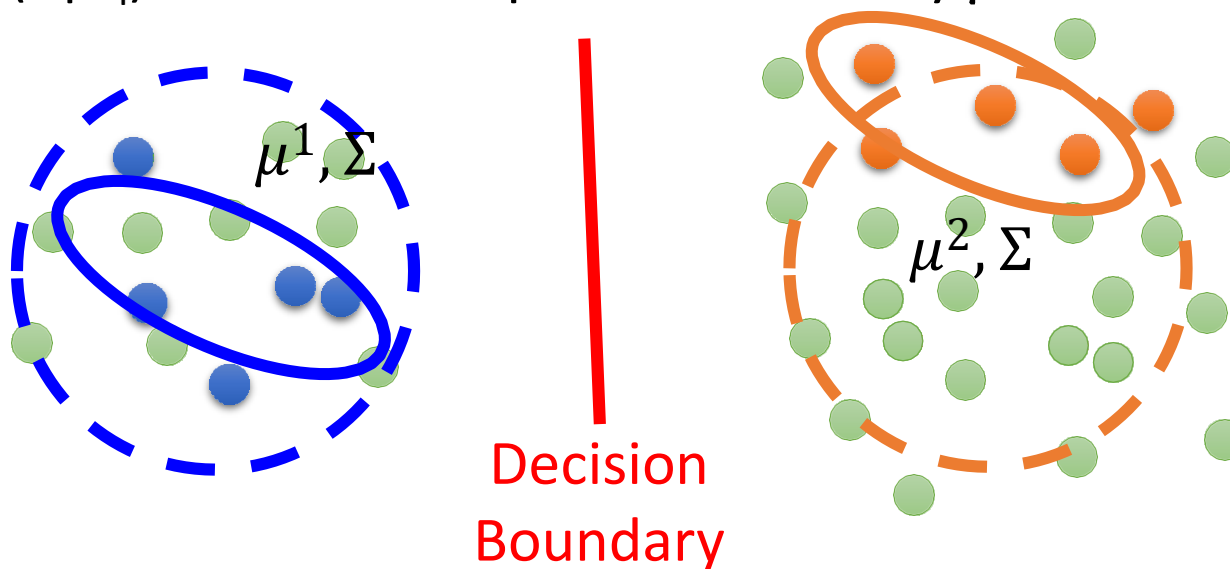
With $P(C_1), P(C_2), \mu^1, \mu^2, \Sigma$

$$P(C_1|x) = \frac{P(x|C_1)P(C_1)}{P(x|C_1)P(C_1) + P(x|C_2)P(C_2)}$$

生成模型



- Given labelled training examples $x^r \in C_1, C_2$
 - looking for most likely prior probability $P(C_i)$ and class-dependent probability $P(x|C_i)$
 - $P(x|C_i)$ is a Gaussian parameterized by μ^i and Σ



The unlabeled data x^u help re-estimate $P(C_1), P(C_2), \mu^1, \mu^2, \Sigma$

生成模型



- Initialization: $\theta = \{P(C_1), P(C_2), \mu^1, \mu^2, \Sigma\}$

E

- Step 1: compute the posterior probability of unlabeled data

$$P_{\theta}(C_1|x^u)$$

Depending on model θ



Back to
step 1

M

- Step 2: update model

$$P(C_1) = \frac{N_1 + \sum_{x^u} P(C_1|x^u)}{N}$$

N : total number of examples
 N_1 : number of examples
belonging to C_1

$$\mu^1 = \frac{1}{N_1} \sum_{x^r \in C_1} x^r + \frac{1}{\sum_{x^u} P(C_1|x^u)} \sum_{x^u} P(C_1|x^u) x^u \dots\dots$$

低密度分离



- Given: labelled data set = $\{(x^r, \hat{y}^r)\}_{r=1}^R$, unlabeled data set = $\{x^u\}_{u=l}^{R+U}$

- Repeat:

- Train model f^* from labelled data set

Independent to the model

Regression?

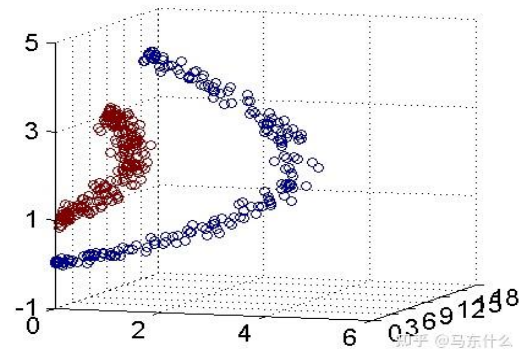
- Apply f^* to the unlabeled data set

- Obtain $\{(x^u, y^u)\}_{u=l}^{R+U}$ Pseudo-label

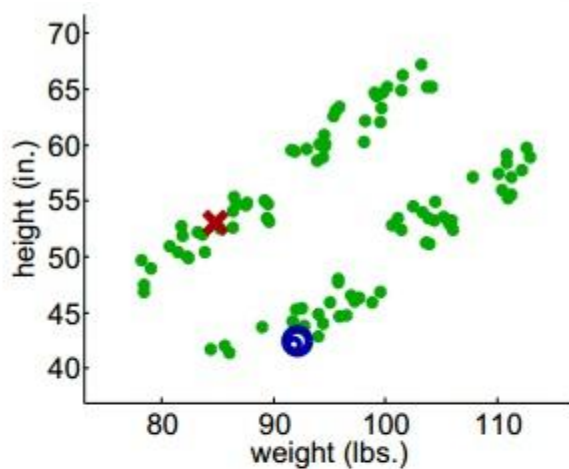
- Remove a set of data from unlabeled data set, and add them into the labeled data set

How to choose the data set remains open

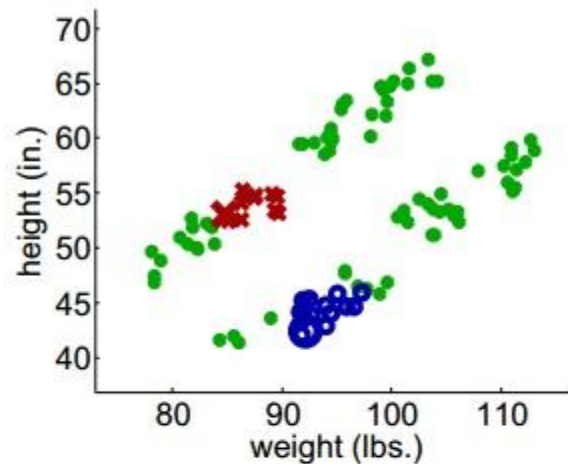
You can also provide a weight to each data.



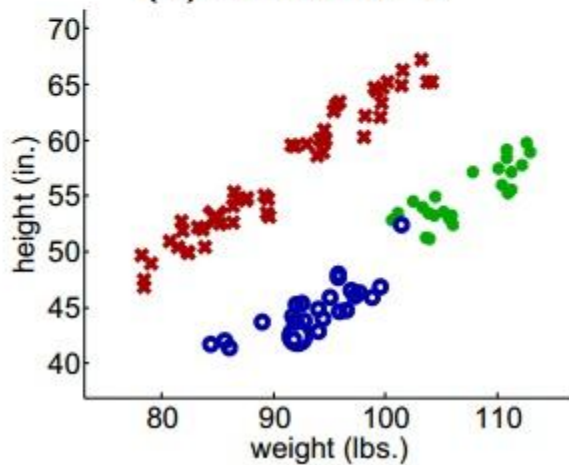
self training



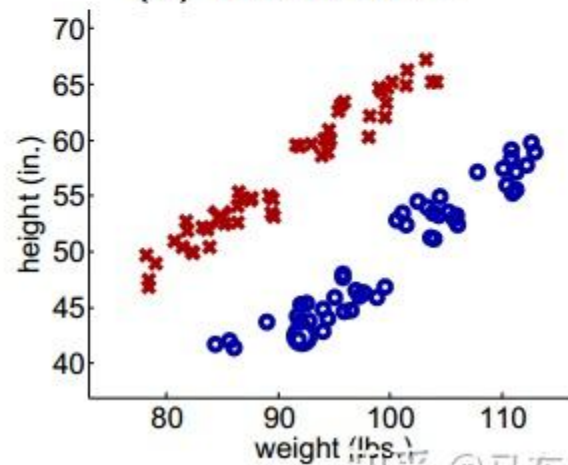
(a) Iteration 1



(b) Iteration 25

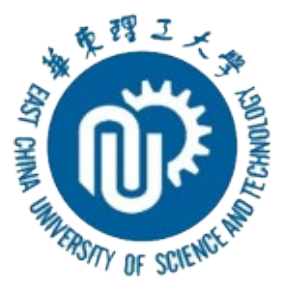


(c) Iteration 74



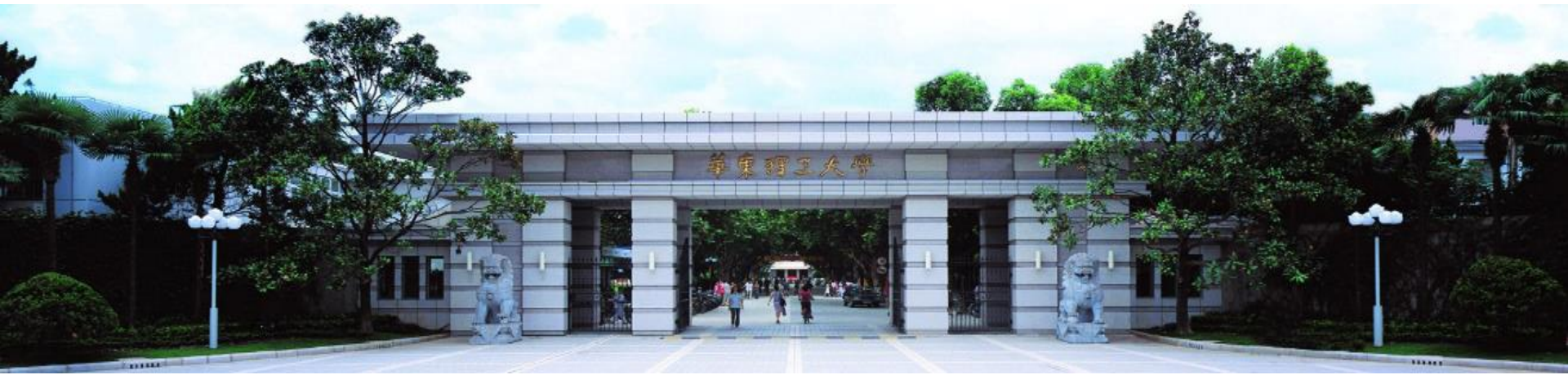
(d) Final labeling of all instances

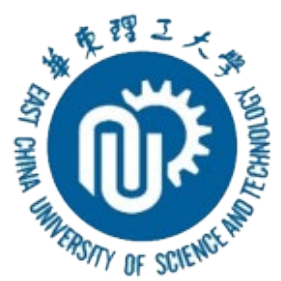
<http://blog.csdn.net/madaxiaomao>



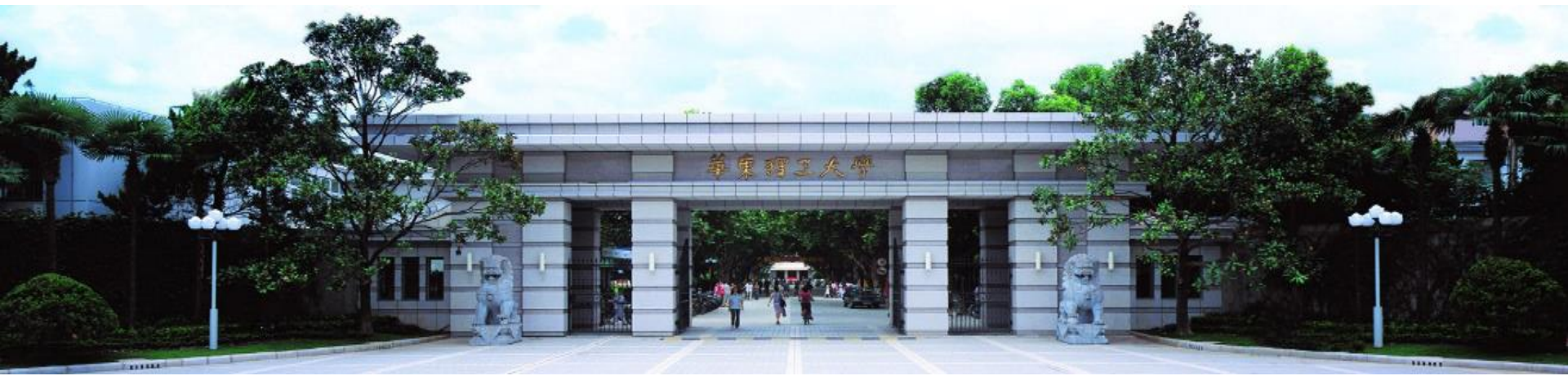
谢谢！

THANK YOU FOR LISTENING





模型独立的学习方式



模型独立的学习方式



- 这些学习方式不限于具体的模型

前馈神经网络、循环神经网络还是其他模型

- 然而一种学习方式往往会对符合某种特性的模型更加青睐

集成学习往往和方差大的模型组合时效果显著。

内容



- 集成学习
- 协同学习
- 多任务学习
- 迁移学习
- 终身学习
- 元学习



三个臭皮匠赛过诸葛亮

定理 10.1: 对于 M 个不同的模型 $f_1(\mathbf{x}), \dots, f_M(\mathbf{x})$, 平均期望错误为 $\bar{\mathcal{R}}(f)$ 。基于简单投票机制的集成模型 $f^{(c)}(\mathbf{x}) = \frac{1}{M} \sum_{m=1}^M f_m(\mathbf{x})$, 其期望错误在 $\frac{1}{M} \bar{\mathcal{R}}(f)$ 和 $\bar{\mathcal{R}}(f)$ 之间。

从定理10.1可知, 为了得到更好的集成效果, 要求每个模型之间具备一定的差异性。并且随着模型数量的增多, 其错误率也会下降, 并趋近于0.



Bagging类

- Bagging (Bootstrap Aggregating) 是一个通过不同模型的训练数据集的独立性来提高不同模型之间的独立性。我们在原始训练集上进行有放回的随机采样，得到 M 个 比较小的训练集并训练 M 个模型，然后通过投票的方法进行模型集成。
- 随机森林 (Random Forest) 是在Bagging的基础上再引入了随机特征，进一步提高每个基模型之间的独立性。在随机森林中，每个基模型都是一棵决策树。



决策树(Decision Tree)

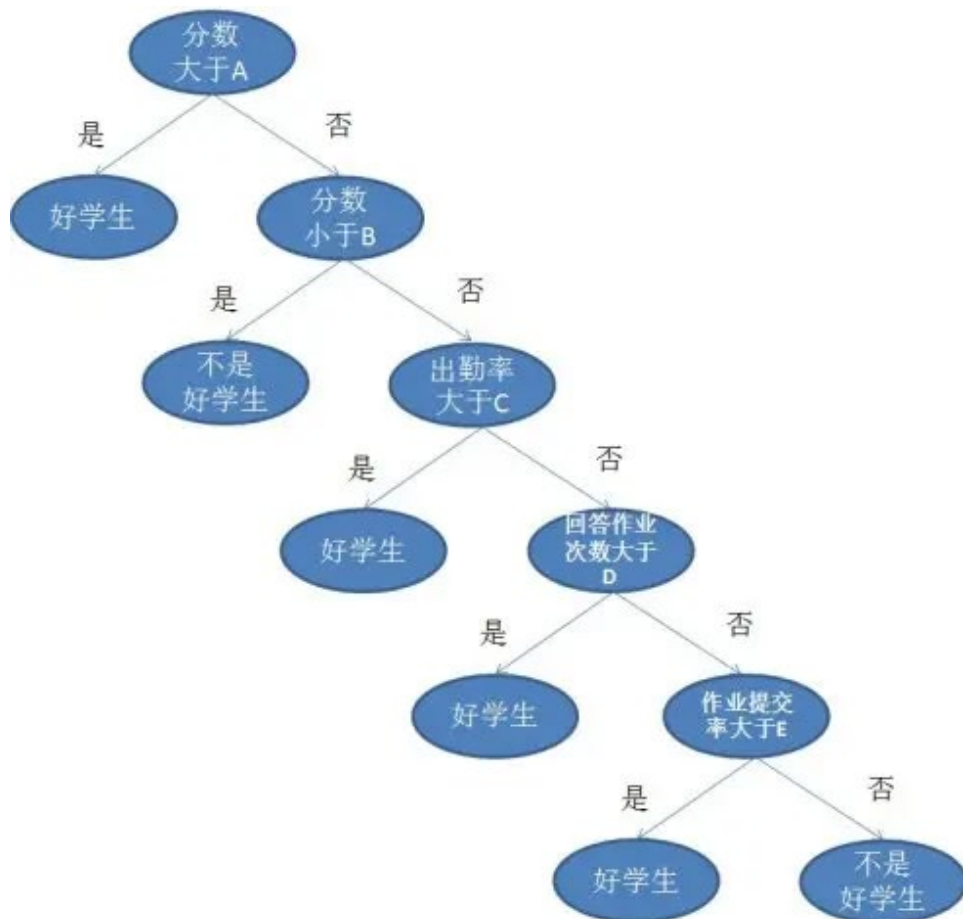
决策树是一种树形结构，其中每个内部节点表示一个属性上的判断，每个分支代表一个判断结果的输出，最后每个叶节点代表一种分类结果。

给出如下的一组数据，一共有十个样本（学生数量），每个样本有分数，出勤率，回答问题次数，作业提交率四个属性，最后判断这些学生是否是好学生。最后一列给出了人工分类结果。

学生编号	分数	出勤率	回答问题次数	作业提交率	分类: 是否好学生
1	99	80%	5	90%	是
2	89	100%	6	100%	是
3	69	100%	7	100%	否
4	50	60%	8	70%	否
5	95	70%	9	80%	否
6	98	60%	10	80%	是
7	92	65%	11	100%	是
8	91	80%	12	85%	是
9	85	80%	13	95%	是
10	85	91%	14	98%	是

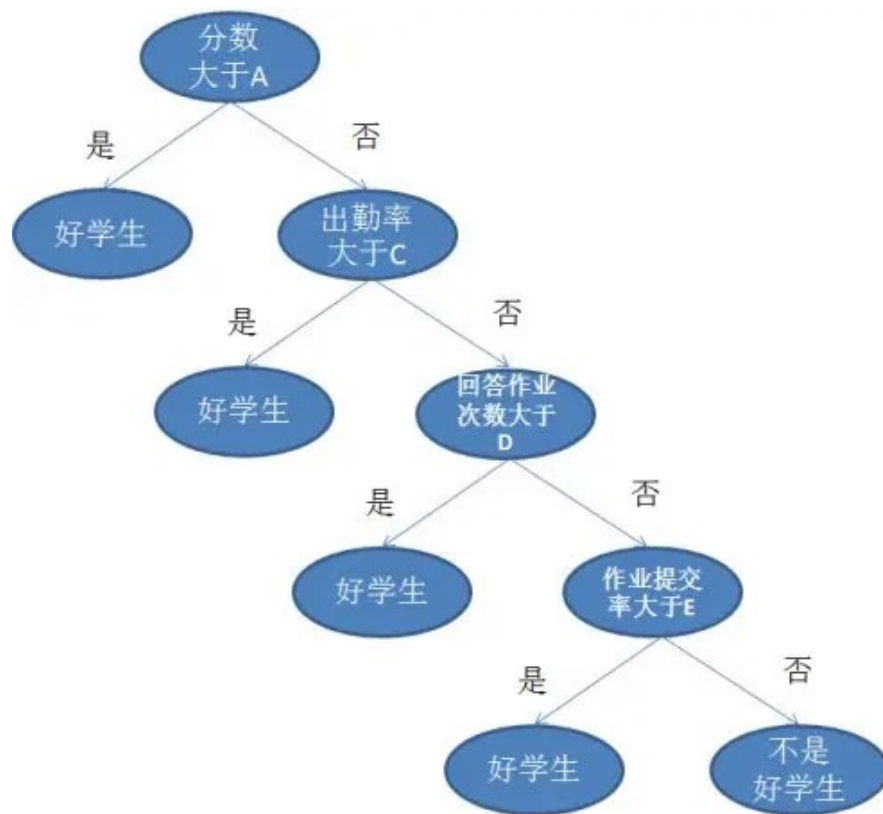
决策树 (Decision Tree)

然后用这一组附带分类结果的样本可以训练出多种多样的决策树，这里为了简化过程，我们假设决策树为二叉树，且类似于下图：



决策树(Decision Tree)

通过学习上表的数据，可以设置A, B, C, D, E的具体值，而A, B, C, D, E则称为阈值。当然也可以有和上图完全不同的树形，比如下图这种的：





所以决策树的生成主要分以下两步，这两步通常通过学习已经知道分类结果的样本来实现。

1. 节点的分裂：一般当一个节点所代表的属性无法给出判断时，则选择将这一节点分成2个

子节点（如不是二叉树的情况会分成 n 个子节点）

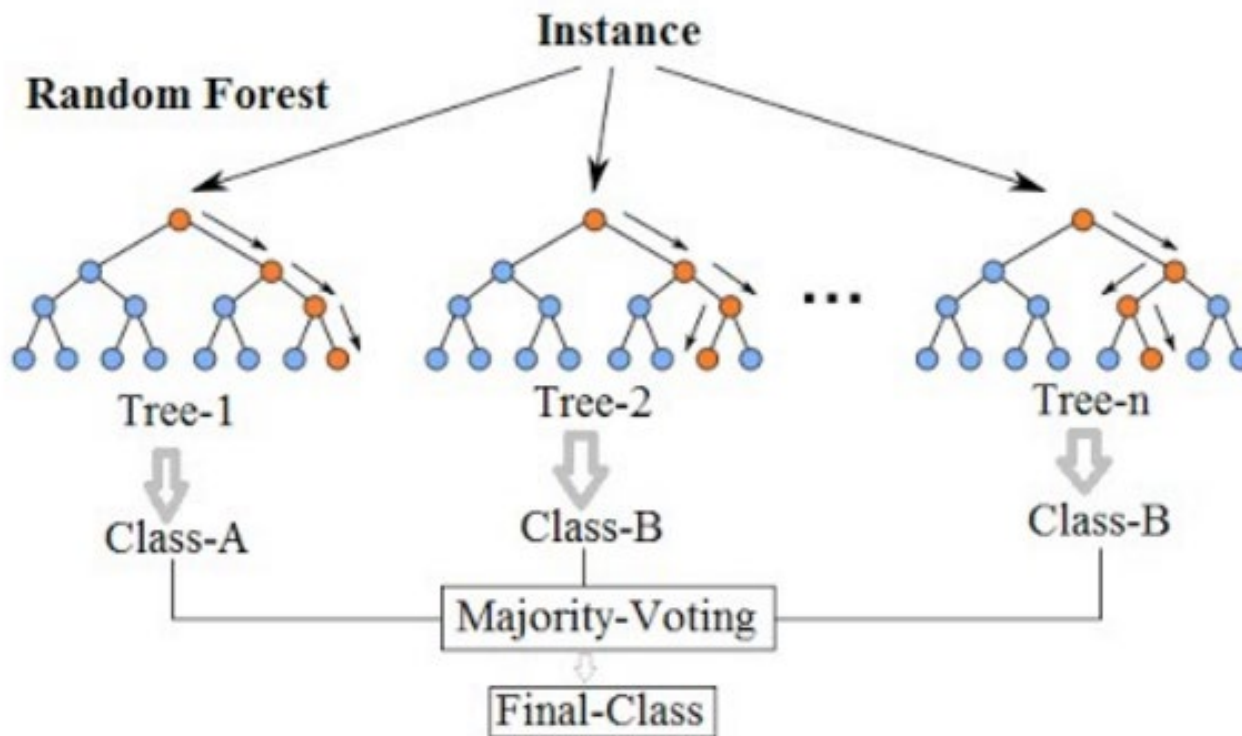
2. 阈值的确定：选择适当的阈值使得分类错误率最小（Training Error）。

比较常用的决策树有ID3，C4.5和CART（Classification And Regression Tree），CART的分类效果一般优于其他决策树。

随机森林



Random Forest Simplified

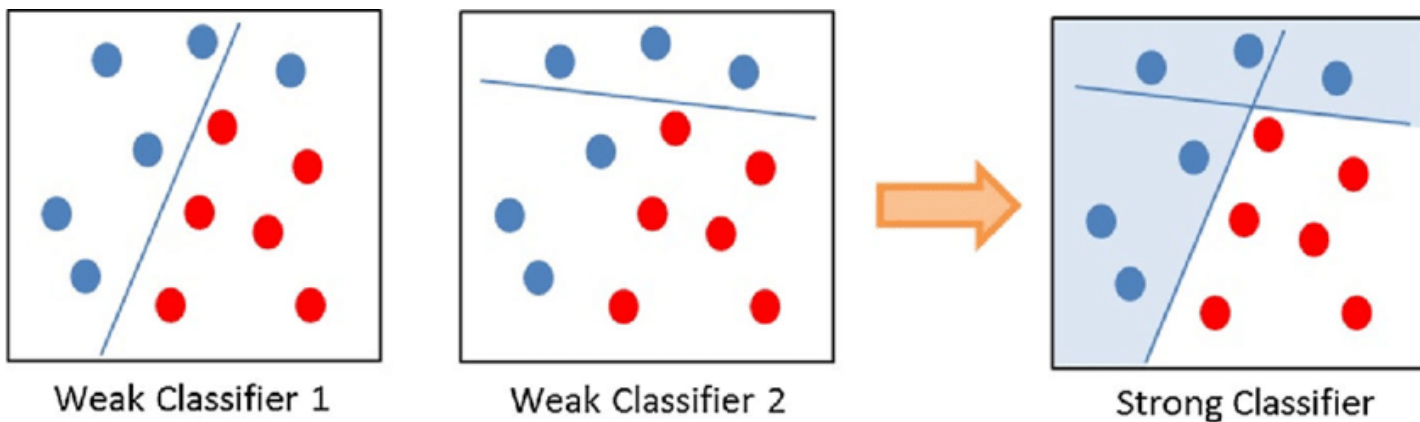


集成方式



Boosting类

- 按照一定的顺序来先后训练不同的基模型，每个模型都针对前序模型的错误进行专门训练。根据前序模型的结果，来调整训练样本的权重，从而增加不同基模型之间的差异性。
- 在随机森林中，每个基模型都是一棵决策树。AdaBoost (Adaptive Boosting) 算法





Boosting类集成模型的目标是学习一个**加性模型** (Additive Model)

$$F(x) = \sum_{m=1}^M \alpha_m f_m(x),$$

其中 $f_m(x)$ 为**弱分类器**(Weak Classifier), 或**基分类器**(Base Classifier), α_m 为弱分类器的集成权重, $F(x)$ 称之为**强分类器**(Strong Classifier)。

Boosting类方法的关键是如何训练每个弱分类器 $f_m(x)$ 及其权重 α_m 。为了提高集成的效果, 应当尽量使得每个弱分类器的差异尽可能大。一种有效的算法是采用迭代的方法来学习每个弱分类器, 即按照一定的顺序依次训练每个弱分类器, 假设已经训练了 m 个弱分类器, 在训练第 $m + 1$ 个弱分类器时, 增加已有弱分类器分错样本的权重, 使得第 $m + 1$ 个弱分类器“更关注”于已有弱分类器分错的样本。这样增加每个弱分类器的差异, 最终提升集成分类器的准确率, 这方法称为**AdaBoost**(Adaptive Boosting) 算法。

AdaBoost



算法 10.1 二分类的 AdaBoost 算法

输入: 训练集 $\{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$, 迭代次数 M

1 初始样本权重: $w_1^{(n)} \leftarrow \frac{1}{N}, \forall n \in [1, N]$;

2 **for** $m = 1 \cdots M$ **do**

3 按照样本权重 $w_m^{(1)}, \dots, w_m^{(N)}$, 学习弱分类器 f_m ;

4 计算弱分类器 f_m 在数据集上的加权错误 ϵ_m ;

5 计算分类器的集成权重:

$$\alpha_m \leftarrow \frac{1}{2} \log \frac{1 - \epsilon_m}{\epsilon_m};$$

6 调整样本权重:

$$w_{m+1}^{(n)} \leftarrow w_m^{(n)} \exp(-\alpha_m y^{(n)} f_m(\mathbf{x}^{(n)})), \forall n \in [1, N];$$

7 **end**

输出: $F(\mathbf{x}) = \text{sgn}\left(\sum_{m=1}^M \alpha_m f_m(\mathbf{x})\right)$



自训练（Self-Training，或 Self-Teaching），也叫自举法（Bootstrapping），是一种非常简单的半监督学习算法。自训练是首先使用标注数据来训练一个模型，并使用这个模型来预测无标注样本的标签，把预测置信度比较高的样本及其预测的伪标签加入训练集，然后重新训练新的模型，并不断重复这个过程。

输入: 标注数据集 $\mathcal{L} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$;

无标注数据集 $\mathcal{U} = \{\mathbf{x}^{(m)}\}_{m=1}^M$;

迭代次数 T ; 每次迭代增加样本数量 P ;

1 for $t = 1 \cdots T$ **do**

2 根据训练集 $\mathcal{L}$, 训练模型 f ;

3 使用模型 f 对无标注数据集 $\mathcal{U}$ 的样本进行预测, 选出预测置信度高的 P
 个样本 $\mathcal{P} = \{(\mathbf{x}^{(p)}, f(\mathbf{x}^{(p)}))\}_{p=1}^P$;

4 更新训练集:

$$\mathcal{L} \leftarrow \mathcal{L} \cup \mathcal{P}, \quad \mathcal{U} \leftarrow \mathcal{U} - \mathcal{P}.$$

5 end

输出: 模型 f

自训练算法通过不断地迭代来提高模型能力。但自训练的缺点是无法保证每次加入训练集的样本的伪标签是正确的。此时如果选择样本的伪标签是错误的，反而会损害模型的预测能力。因此，自训练算法最关键的步骤是如何设置挑选样本的标准。



Multi-View Learning

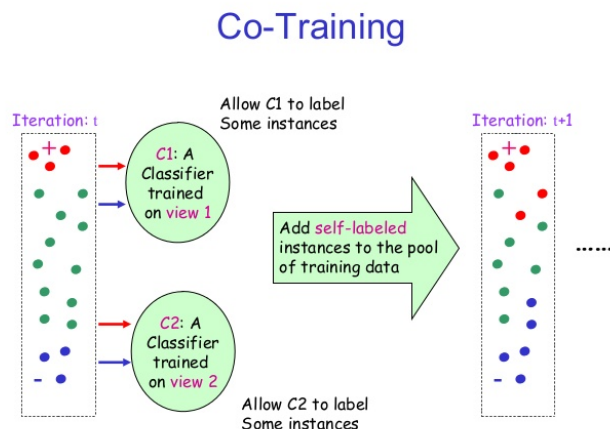
协同训练（co-training）算法是一种多学习器的半监督学习算法。它是多视图（multi-view）学习的代表。以电影为例，它拥有多个属性集：图像、声音、字幕等。每个属性集就构成了一个视图。协同训练假设不同的视图具有“相容性”。

所谓相容性是指，如果一个电影从图像判断，像是动作片，那么从声音判断，也有很大可能是动作片。

协同训练的算法流程：

1. 在每个视图上，基于有标记数据，分别训练出一个分类器。
2. 每个分类器挑选自己最有把握的未标记数据，赋予伪标记。
3. 其他分类器利用伪标记，进一步训练模型，最终达到相互促进的目的。

理论上，如果不同视图之间具有完全相容性，则模型准确度可达到任意上限。而实际中，虽然很难满足完全相容性假设，但该算法仍能有效提升模型的准确度。





算法 10.3 协同训练的训练过程

输入: 标注数据集 $\mathcal{L} = \{(\mathbf{x}^{(n)}, y^{(n)})\}_{n=1}^N$;

无标注数据集 $\mathcal{U} = \{\mathbf{x}^{(m)}\}_{m=1}^M$;

迭代次数 T ; 候选池大小 K ; 每次迭代增加样本数量 $2P$;

1 **for** $t = 1 \cdots T$ **do**

2 根据训练集 $\mathcal{L}$ 的视角 V_1 训练训练模型 f_1 ;

3 根据训练集 $\mathcal{L}$ 的视角 V_2 训练训练模型 f_2 ;

4 从无标注数据集 $\mathcal{U}$ 上随机选取一些样本放入候选池 $\mathcal{U}'$, 使得 $|\mathcal{U}'| = K$;

5 **for** $f \in f_1, f_2$ **do**

6 使用模型 f 预测候选池 $\mathcal{U}'$ 中的样本的伪标签;

7 **for** $p = 1 \cdots P$ **do**

8 根据标签分布, 随机选取一个标签 y ;

9 从 $\mathcal{U}'$ 中选出伪标签为 y , 并且预测置信度最高的样本 $\mathbf{x}$;

10 更新训练集:

$$\mathcal{L} \leftarrow \mathcal{L} \cup \{(\mathbf{x}, y)\}, \quad \mathcal{U}' \leftarrow \mathcal{U}' - \{(\mathbf{x}, y)\}.$$

11 **end**

12 **end**

13 **end**

输出: 模型 f_1, f_2

多任务学习

Multitask Learning



多任务学习 (Multi-task Learning)是指同时学习多个相关任务，让这些任务在学习过程中共享知识，利用多个任务之间的相关性来改进模型在每个任务上的性能和泛化能力。

多任务学习可以看作一种**归纳迁移学习**(Inductive Transfer Learning)，即通过利用包含在相关任务中的信息作为归纳偏置(Inductive Bias)来提高泛化能力。

单任务学习 (single task learning)：一个loss，一个任务，例如NLP里的情感分类、NER任务一般都是可以叫单任务学习。

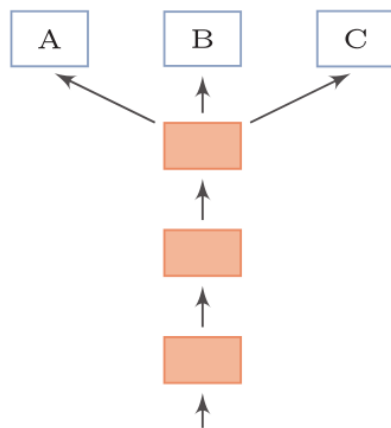




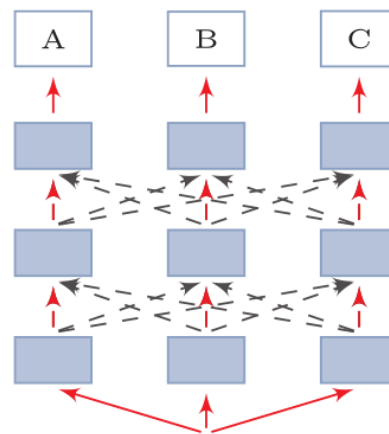
多任务学习 (Multi-task Learning)是指同时学习多个相关任务，让这些任务在学习过程中共享知识，利用多个任务之间的相关性来改进模型在每个任务上的性能和泛化能力。

多任务学习可以看作一种**归纳迁移学习**(Inductive Transfer Learning)，即通过利用包含在相关任务中的信息作为归纳偏置(Inductive Bias)来提高泛化能力。

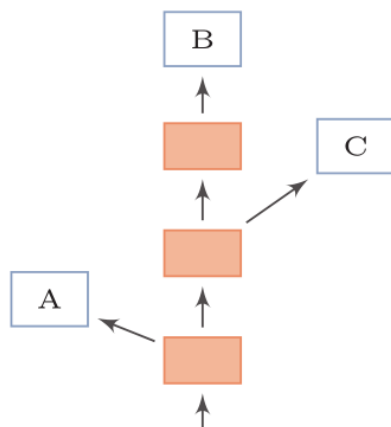
共享机制 多任务学习的主要挑战在于如何设计多任务之间的共享机制，在传统的机器学习算法中，引入共享的信息是比较困难的，通常会导致模型变得复杂，但是在神经网络模型中，模型共享变得相对比较容易，深度神经网络模型提供了一种很方便的信息共享方式，可以很容易地进行多任务学习。多任务学习的共享机制比较灵活，有很多种共享模式。



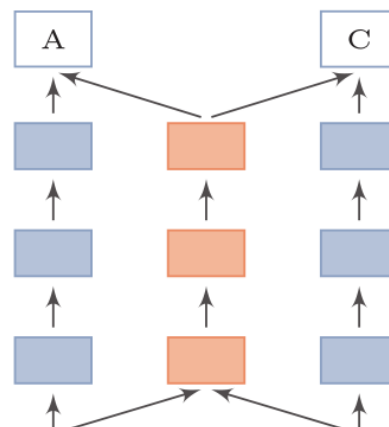
(a) 硬共享模式



(b) 软共享模式



(c) 层次共享模式



(d) 共享-私有模式

其中A、B和C表示三个不同的任务，红色框表示共享模块，蓝色框表示任务特定模块。

多任务学习中的四种常见的共享模式

Multitask Learning*

Machine Learning 1997

RICH CARUANA

caruana@cs.cmu.edu

School of Computer Science, Carnegie Mellon University, Pittsburgh, PA 15213

Multitask Learning is an approach to **inductive transfer** that improves **generalization** by using the domain information contained in the training signals of related tasks as an **inductive bias**. It does this by learning tasks in parallel while using a **shared representation**; what is learned for each task can help other tasks be learned better.

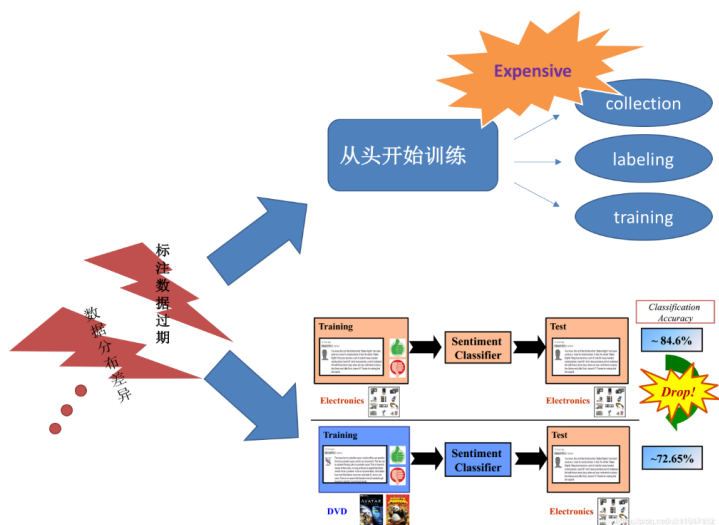
多任务学习是一种归纳迁移方法，它通过使用相关任务训练信号中包含的领域信息作为归纳偏差来提高泛化能力。它通过在使用共享表示的同时并行学习任务来做到这一点；从每项任务中学到的东西可以帮助更好地学习其他任务。



在有监督的机器学习和尤其是深度学习的场景应用中，需要大量的标注数据。标注数据是一项枯燥无味且花费巨大的任务，关键是现实场景中，往往无法标注足够的数据。而且模型的训练是极其耗时的。因此迁移学习应运而生。

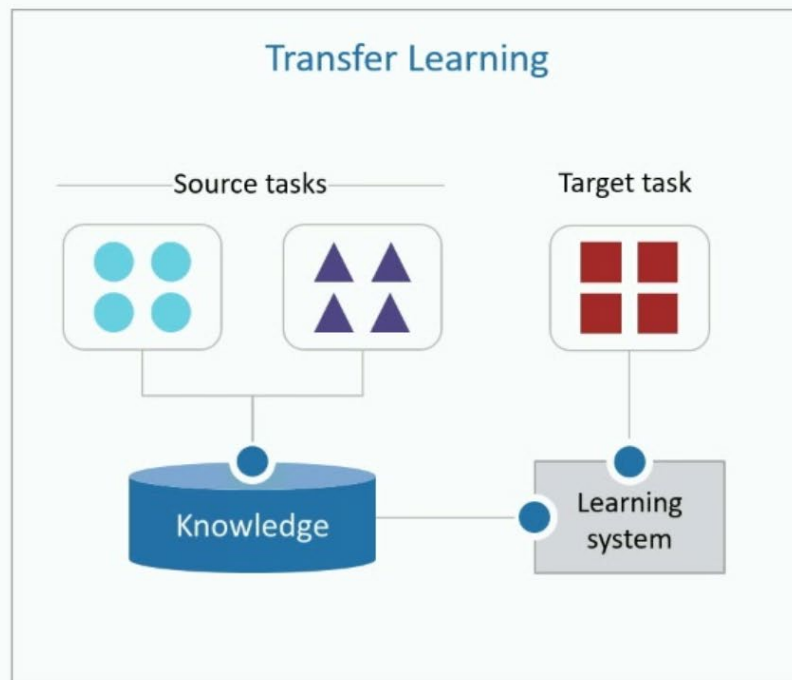
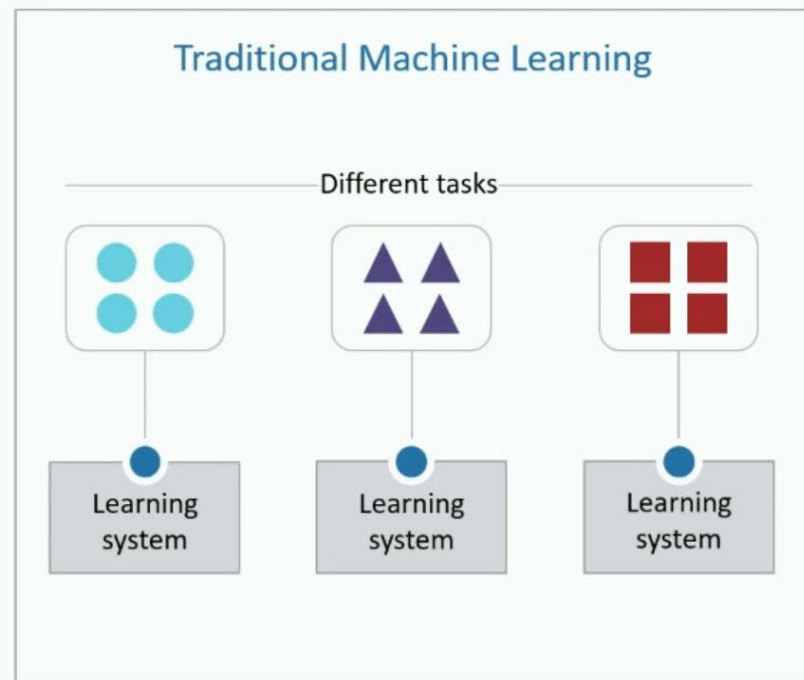
传统机器学习(主要指监督学习)：基于同分布假设；需要大量标注数据
然而实际使用过程中不同数据集可能存在一些问题，比如：

- (1) 数据分布差异
- (2) 标注数据过期
- (3) 训练数据过期，也就是好不容易标定的数据要被丢弃，有些应用中数据是分布随着时间推移会有变化，将以往数据尽可能的利用起来，是迁移学习的思想。





迁移学习是指两个不同领域的知识迁移过程，利用源领域（Source Domain） D_S 中学到的知识来帮助目标领域（Target Domain） D_T 上的学习任务，源领域的训练样本数量一般远大于目标领域。



迁移学习

Transfer Learning



举一个很直观的例子，我们人学习在学习完自行车之后，去学电动车就会相对容易，因为人类能够自然而然地将自行车上学习到的知识灵活的应用到电动车驾驶的学习上，并且稍作调整就能很好的掌握电动车驾驶技术，而在算法层面，迁移学习就是帮助模型能够像人类一样自然而然的将知识进行跨领域的迁移。



比如，我们有时候在某个感兴趣的领域有个分类任务，但是我们只有另一个感兴趣领域的足够训练数据，并且后者的数据可能处于与之前领域不同的特征空间或者遵循不同的数据分布。这类情况下，如果知识的迁移做的成功，我们将会通过避免花费大量昂贵的标记样本数据的代价，使得学习性能取得显著的提升。

这类问题在文本和图像领域非常常见，例如经典的**bert**，谷歌使用了**transformer**，训练了海量的预料数据，然后以**bert**——预训练模型的形式，帮助常规的文本分类问题进行文本向量化，这比我们使用自己少量的语料进行训练得到的模型效果要好得多。



什么时候我们需要用到迁移学习？

数据量很大但是标注样本很少或者数据量很小（因此标注样本也很小）

这是我们日常工作中非常常见的问题，无论是cv、nlp还是tabular，都会有数据量少或者标注少或者二者都很少的问题。

大数据与弱计算之间的矛盾

最典型的例子就是Bert、ResNet这些巨无霸模型，普通公司很难有那么强的算力（即使有钱也不舍得花）去自己训练一个通用的大型模型，这个时候通过迁移学习，比如直接使用google开源的bert、resnet来对文本进行快速编码从而快速的解决文本分类或者文本聚类的问题，效果要比自己在自己少量的数据上折腾效果好的太多，目前这也是很多nlp相关的projects的常见做法，cv中也大都是这么来做的。

需要一个公共的普适化模型帮助满足人们的个性化需求

典型的例子就是cv和nlp领域常常听到的fine-tune了，谷歌给nlper提供了伟大的bert，算法工程师们针对各自面对的不同的问题对bert进行fine-tune或者其它改造来满足自身的个性化需求，说白了，就是你下载别人预训练的模型直接使用的效果存在一定的误差，但是针对于自身的数据做一些fine-tuning就可以较好的满足自身的个性化的需求，在自己的任务上得到比较好的模型性能提升。



迁移学习，并不是某一类特定的算法，而是一种处理问题的思想。

具体迁移学习往往分为两个步骤：

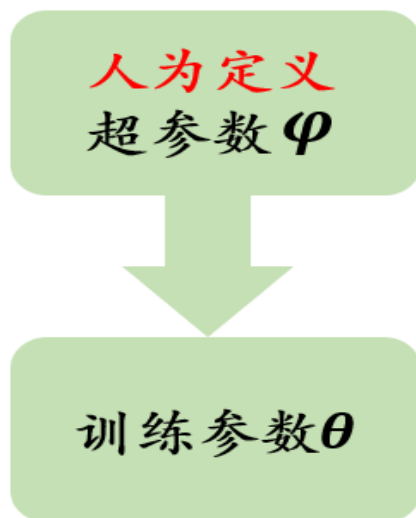
- 1.根据超大规模数据，对模型进行训练；
- 2.根据具体场景任务 进行微调（可以微调权重，还可以调整终端的结构）

根据特征空间和迁移方法，可以将迁移学习分为不同的种类。根据特征空间和标签空间是否相同，可将迁移学习分为 异构迁移学习 和 同构迁移学习。



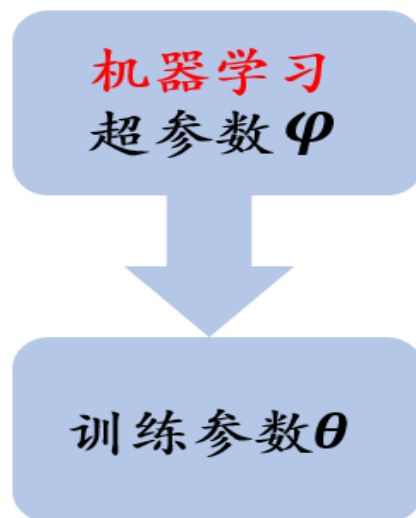
元学习希望使得模型获取一种学会学习调参的能力，使其可以在获取已有知识的基础上快速学习新的任务。机器学习是先人为调参，之后直接训练特定任务下深度模型。元学习则是先通过其它的任务训练出一个较好的超参数，然后再对特定任务进行训练。

Machine Learning



VS

Meta Learning



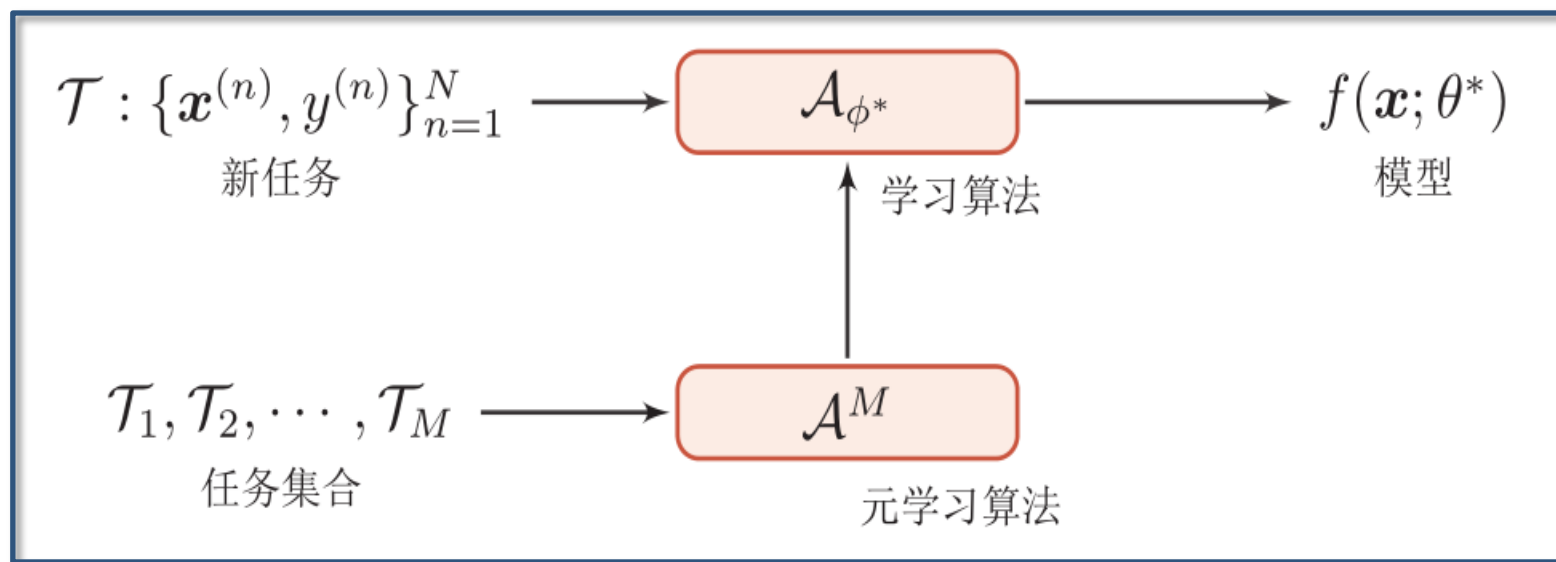
Andrychowicz M, Denil M, Gomez S, et al., 2016. Learning to learn by gradient descent by gradient descent[C]//Advances in Neural Information Processing Systems. 3981-3989.



在机器学习中，训练单位是样本数据，通过数据来对模型进行优化；数据可以分为训练集、测试集和验证集。

在元学习中，训练单位是任务，一般有两个任务分别是训练任务（Train Tasks）亦称跨任务（Across Tasks）和测试任务（Test Task）亦称单任务（Within Task）。

训练任务要准备许多子任务来进行学习，目的是学习出一个较好的超参数，测试任务是利用训练任务学习出的超参数对特定任务进行训练。训练任务中的每个任务的数据分为 Support set 和 Query set；Test Task 中数据分为训练集和测试集。



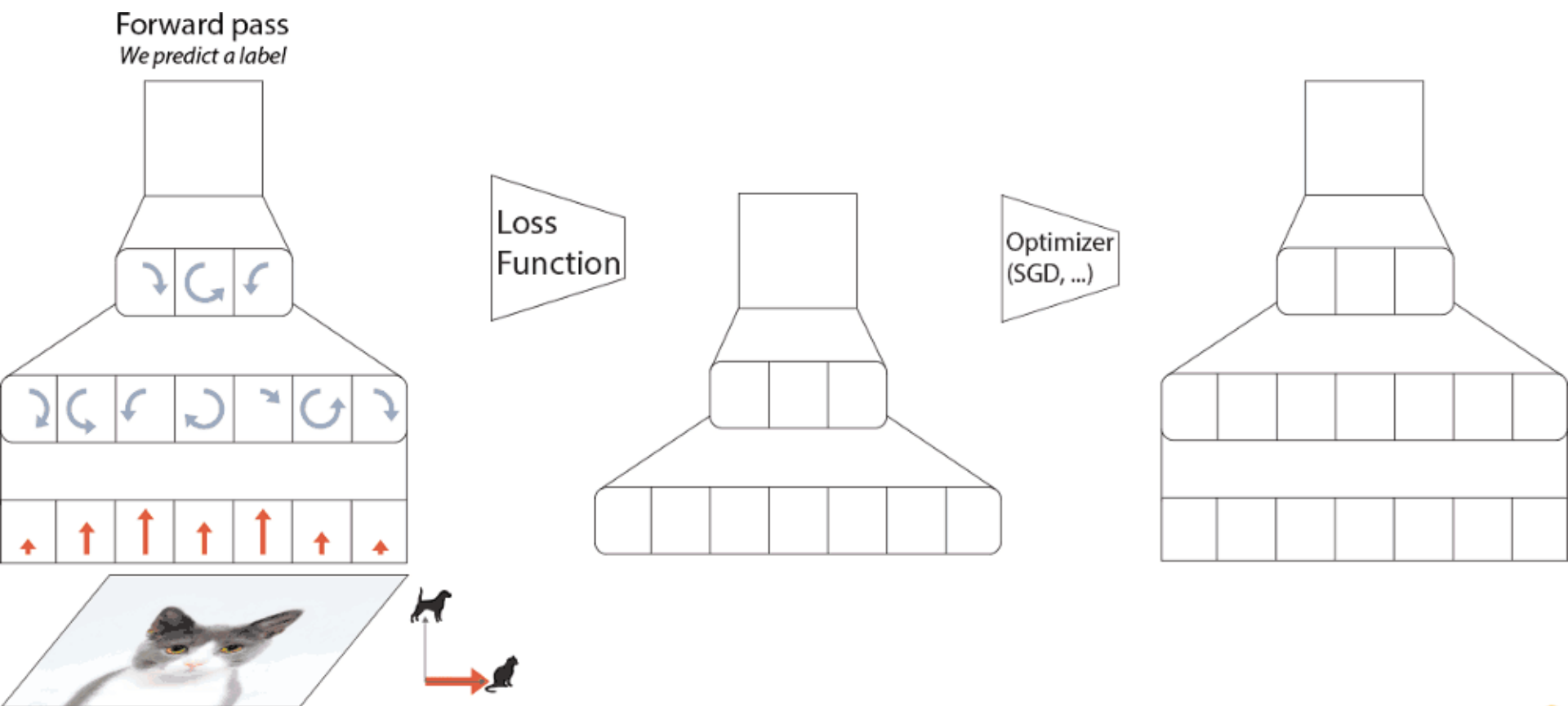
元学习

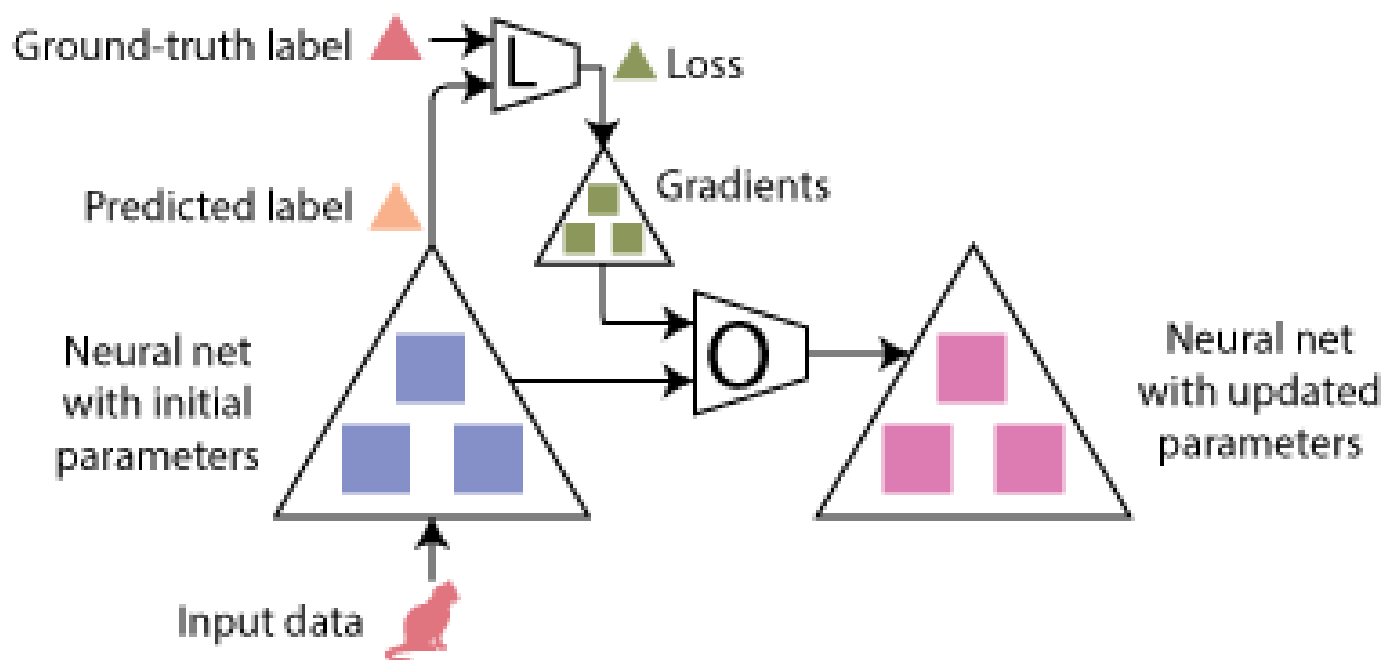
Meta Learning



元学习（**Meta-learning**）是机器学习领域一个令人兴奋的研究趋势，它解决了学会学习（**learning to learn**）的问题。

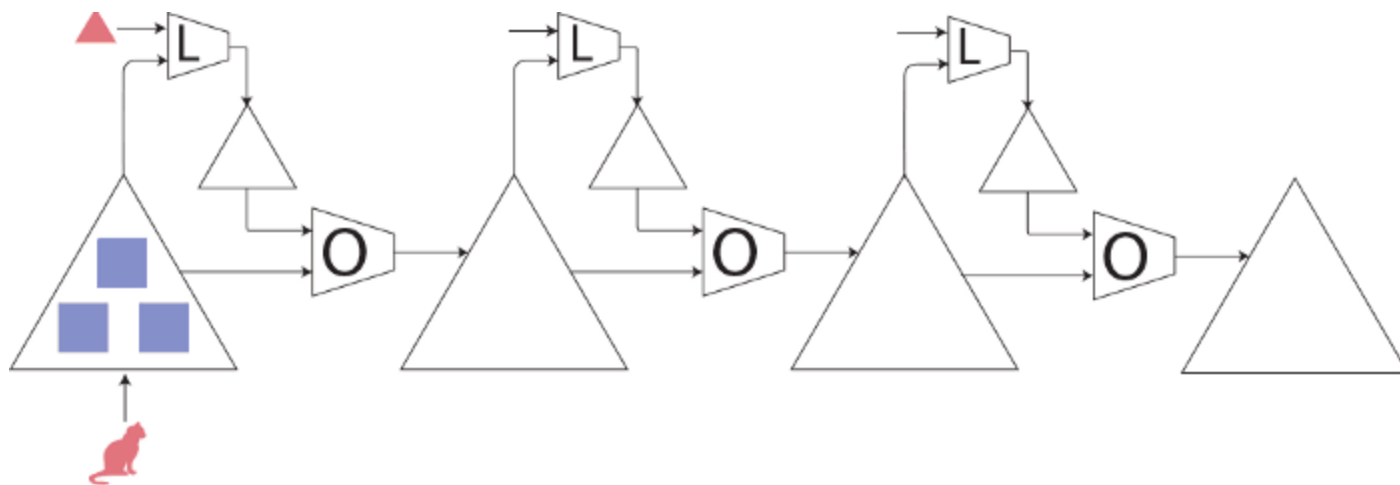
机器学习研究的传统模式是获取特定任务的庞大数据集，并利用该数据集从头开始训练模型。显然，这与人类如何利用过去的经验快速学习新任务相去甚远。那是因为人类学会了学习（**learn to learn**）。





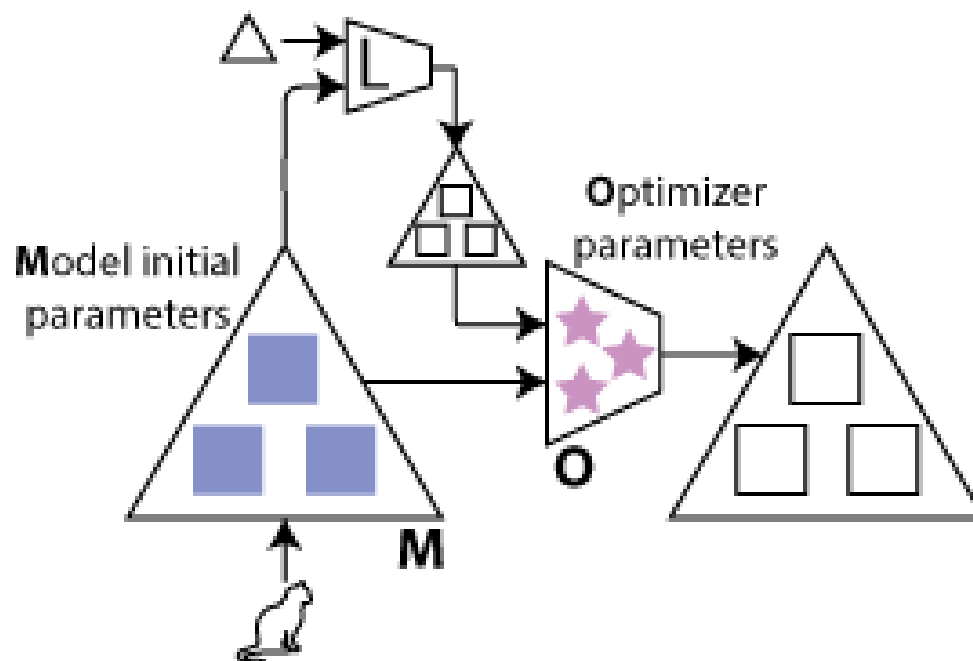
元学习

Meta Learning



元学习

Meta Learning



元学习

Meta Learning

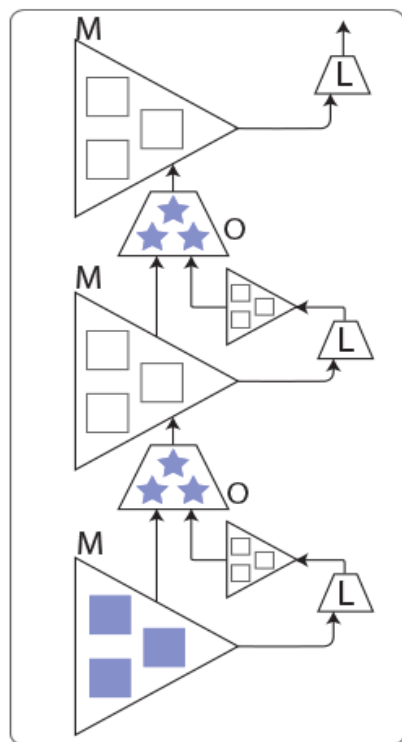


模型（M）的模型是我们以前的神经网络。现在可以将其视为低级别的网络（low-level network）。有时称为优化器（optimizer）或学习器（learner）。模型的权重为图中的■。

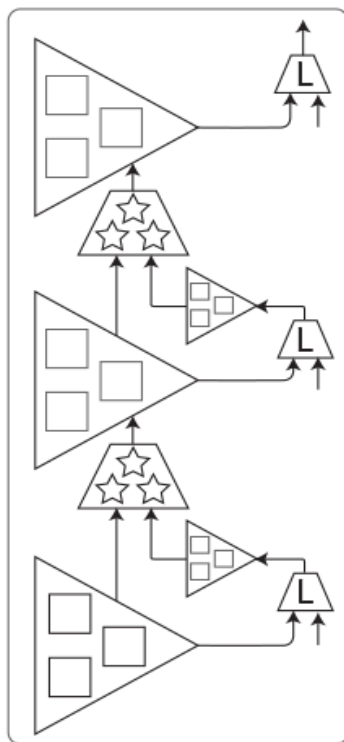
优化器（O）或元学习器（meta-learner）是一个更高级别的模型（higher-level model），它正在更新较低级别网络（模型）的权重。优化器的权重为上图中的★。

Meta-forward pass

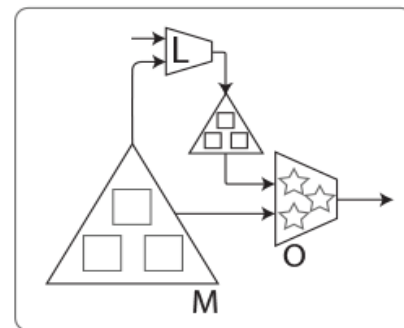
We train the Model



Meta-Loss



Meta-Optimizer

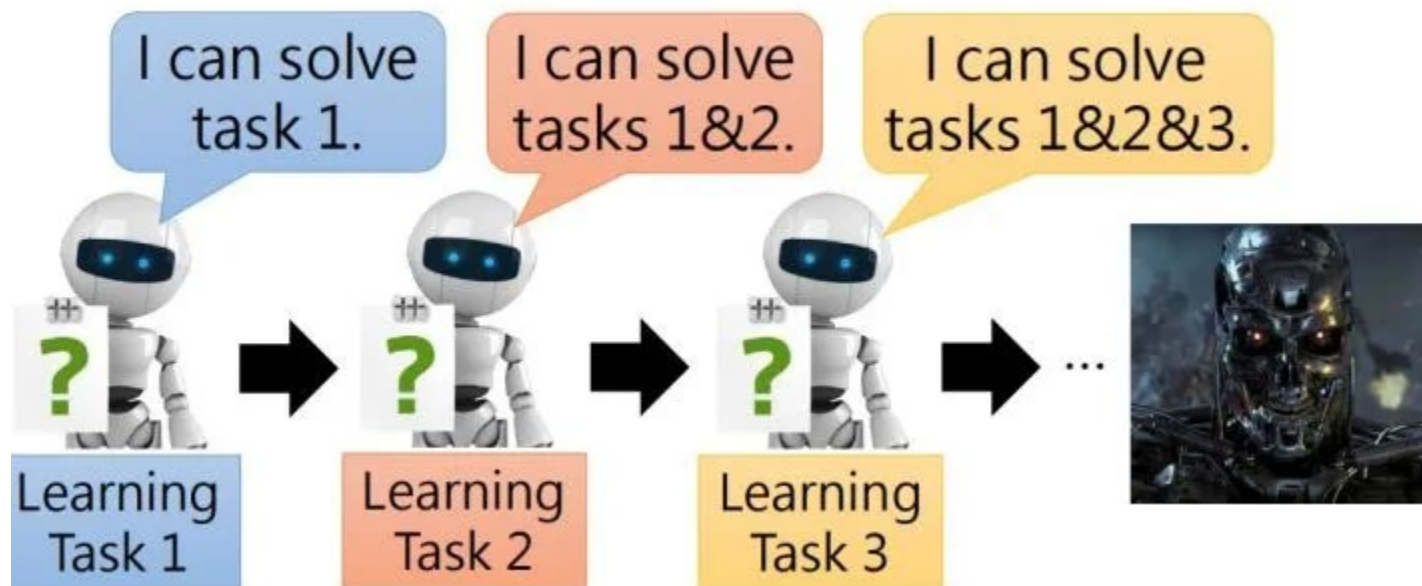


终身学习

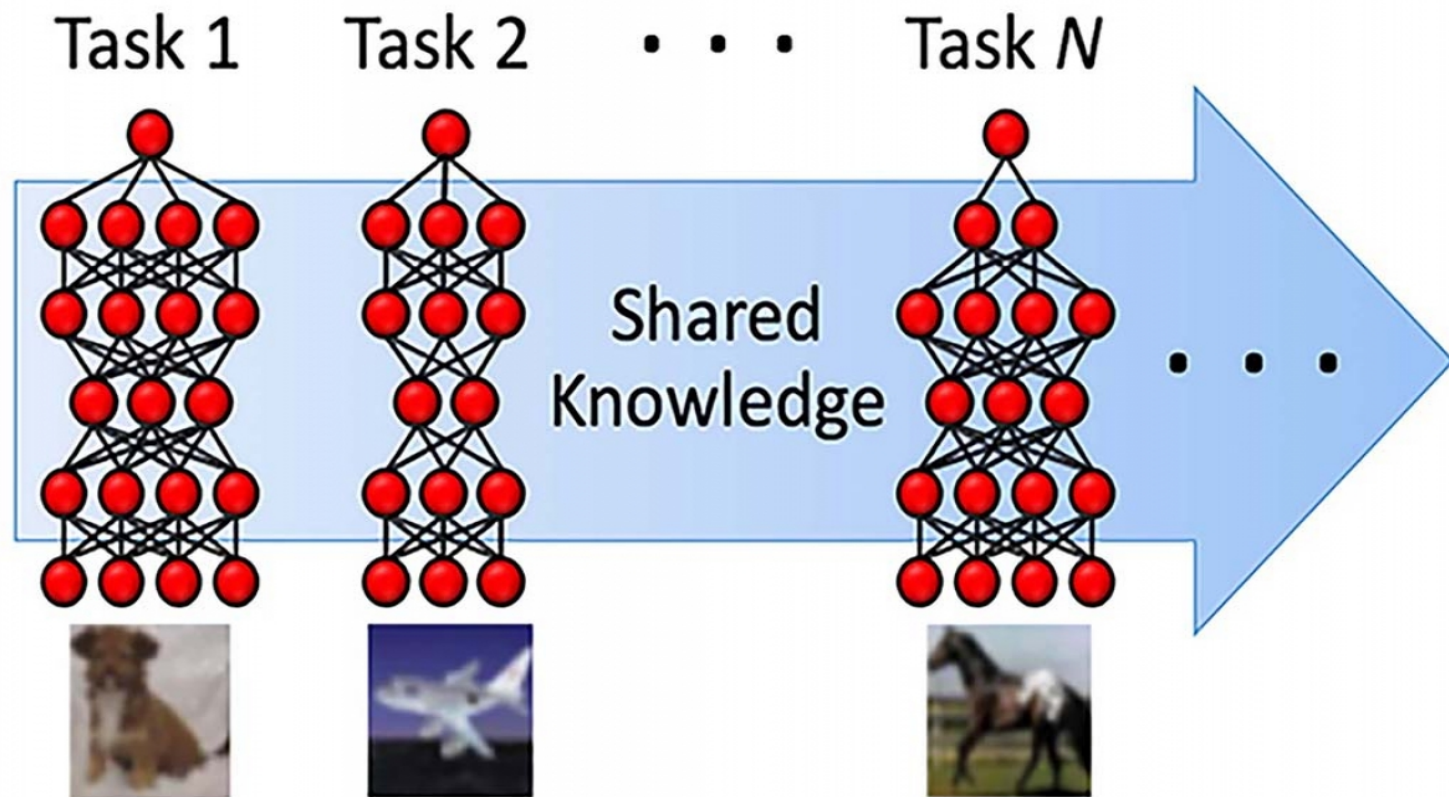
Lifelong Learning



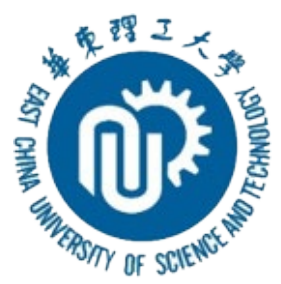
Knowledge Retention (but Not Intransigence) and Knowledge Transfer 知识保留(但不妥协)及知识迁移。这意味着以前学到的知识，需要能够促进下一个任务Task B的学习；学完Task B之后，要不仅可以很好得处理Task B, 同时在Task A 上不能有明显的下降，最好Task A的性能，也能够有所提升或者是保持。



终身学习



终身学习（Lifelong Learning），也叫**持续学习**（Continuous Learning）是指像人类一样具有持续不断的学习能力，根据历史任务中学到的经验和知识来帮助学习不断出现的新任务，并且这些经验和知识是持续累积的，不会因为新的任务而忘记旧的知识。



谢谢！

THANK YOU FOR LISTENING

